■ **The Ultimate Graph Algorithms Handbook (Pattern-Based Edition)**

How This Book Works

For **EVERY** topic in graph, I will break it down into **patterns**. Each pattern has:

- **Theory** (what, why, when, properties, time complexity)

- **Visual Diagram** (see how it works)

- **Template Code** (ready to use)

- **10 Practice Questions** (5 LeetCode + 5 GFG, categorized by pattern)

PART 1: UNION-FIND (DISJOINT SET UNION)

Why DSU Deserves Its Own Deep Dive

DSU is **not just one algorithm** — it's a **family of patterns**. Each pattern solves a different type of problem. If you don't learn the patterns, you'll struggle to recognize when to apply DSU.

■ **DSU: Properties & Complexity**

Property	Value
----------	-------
Time Complexity (Find)	O(α(n)) ≈ O(1) with path compression
Time Complexity (Union)	O(α(n)) ≈ O(1) with union by rank
Space Complexity	O(n)
Works On	Undirected Graphs only
α(n)	Inverse Ackermann function (≤ 4 for all practical n)

When to Use DSU (The Signals)

Problem Signal	DSU Pattern
----------	----------
"Find connected components"	Pattern 1
"Merge groups/sets"	Pattern 1
"Find redundant/extra edge"	Pattern 2
"Add edges and query connectivity"	Pattern 3
"Answer queries offline with thresholds"	Pattern 4
"Lexicographically smallest after swaps"	Pattern 5
"Graph connectivity with two players"	Pattern 6
"Union elements by factors/properties"	Pattern 7
"Satisfy equality/inequality equations"	Pattern 8
"Find when two nodes become connected"	Pattern 9
"Track component sizes"	Pattern 10

PATTERN 1: Basic Connectivity & Component Counting

■ Theory

The **most fundamental DSU pattern**. You have a graph. You need to:

- Find how many connected components exist

- Check if two nodes are in the same component

- Merge components as edges are added

Properties:

- After all unions, each component has exactly **one root**

- Number of components = number of nodes with `parent[i] == i`

- Union operation reduces component count by 1 each time

■ Visual Diagram

```
Initial: 5 isolated nodes
[0] [1] [2] [3] [4]  → 5 components

Union(0,1):
[0-1] [2] [3] [4]    → 4 components
```

```
Union(2,3):
[0-1] [2-3] [4]      → 3 components

Union(1,2):
[0-1-2-3] [4]        → 2 components

Find(0) == Find(3)? Yes (same component)
```

■ Template Code

```
class DSU {
    int[] parent;
    int[] rank;
    int components;

    public DSU(int n) {
        parent = new int[n];
        rank = new int[n];
        components = n;
        for (int i = 0; i < n; i++) parent[i] = i;
    }

    public int find(int x) {
        if (parent[x] != x) {
            parent[x] = find(parent[x]);  // Path compression
        }
        return parent[x];
    }

    public boolean union(int x, int y) {
        int px = find(x), py = find(y);
        if (px == py) return false;

        // Union by rank
        if (rank[px] < rank[py]) {
            parent[px] = py;
        } else if (rank[px] > rank[py]) {
            parent[py] = px;
        } else {
            parent[py] = px;
            rank[px]++;
        }
        components--;
        return true;
    }

    public boolean connected(int x, int y) {
        return find(x) == find(y);
    }

    public int countComponents() {
        return components;
    }
}
```

■ 10 Practice Questions

Provinces](https://leetcode.com/problems/number-of-provinces/)
Connected Components in Undirected Graph](https://leetcode.com/problems/number-of-connected-components-in-an-undirected-grap
les](https://www.geeksforgeeks.org/problems/friend-circles/0)
nectivity](https://www.geeksforgeeks.org/problems/connecting-the-graph/1)
umber of Islands Using DSU](https://www.geeksforgeeks.org/number-of-islands-using-disjoint-set/)
es Removed with Same Row or Column](https://leetcode.com/problems/most-stones-removed-with-same-row-or-column/)
Operations to Make Network Connected](https://leetcode.com/problems/number-of-operations-to-make-network-connected/)
eachable Pairs of Nodes in Undirected Graph](https://leetcode.com/problems/count-unreachable-pairs-of-nodes-in-an-undirected-graph/
t Union Implementation](https://www.geeksforgeeks.org/problems/disjoint-set-union-find/1)
Rank and Path Compression](https://www.geeksforgeeks.org/problems/union-find/1)

PATTERN 2: Cycle Detection in Undirected Graph

■ Theory

In an **undirected graph**, adding an edge creates a cycle **if and only if** its two endpoints are already in the same connected component.

Properties:

- Process edges one by one

- Before union, check if `find(u) == find(v)`

- If yes → this edge creates a cycle

- This is how Kruskal's MST algorithm detects cycles

■ Visual Diagram

```
Edges processed in order:

1. Add edge (0-1): 0 and 1 in different components → union ■
   Components: {0,1}, {2}, {3}

2. Add edge (1-2): 1 in {0,1}, 2 in {2} → different → union ■
   Components: {0,1,2}, {3}

3. Add edge (0-2): 0 and 2 in SAME component {0,1,2} → CYCLE! ■
   Edge (0-2) is redundant
```

■ Template Code

```
public int[] findRedundantConnection(int[][] edges, int n) {
    DSU dsu = new DSU(n);
    for (int[] edge : edges) {
        int u = edge[0], v = edge[1];
        if (dsu.find(u) == dsu.find(v)) {
            return edge;  // This edge creates a cycle
        }
        dsu.union(u, v);
    }
    return new int[0];
}
```

■ 10 Practice Questions

ant Connection](https://leetcode.com/problems/redundant-connection/)
ant Connection II](https://leetcode.com/problems/redundant-connection-ii/)
ycle in Undirected Graph](https://www.geeksforgeeks.org/problems/detect-cycle-in-an-undirected-graph/1)
alid Tree](https://leetcode.com/problems/graph-valid-tree/)
ath Exists in Graph](https://leetcode.com/problems/find-if-path-exists-in-graph/)
etection in Undirected Graph using DSU](https://www.geeksforgeeks.org/union-find-cycle-detection-union-find/)
Number of Swaps to Make a String Balanced](https://www.geeksforgeeks.org/minimum-number-of-swaps-to-make-a-string-balanced/)
a Given Graph is Tree or Not](https://www.geeksforgeeks.org/check-given-graph-tree/)
of Ways to Reconnect Graph](https://www.geeksforgeeks.org/number-of-ways-to-reconnect-graph/)
Cost to Connect Sticks](https://www.geeksforgeeks.org/minimum-cost-to-connect-sticks/)

PATTERN 3: Offline Queries (Sort + Process Incrementally)

■ Theory

This is where DSU **truly shines**. When you have queries asking "is there a path with all edges weight < limit?", you can't answer online efficiently. Instead:

1. **Sort edges by weight**
2. **Sort queries by limit**
3. **Process queries in increasing order** - add edges gradually
4. **Answer query** using DSU connectivity check

Properties:

- Time Complexity: O(E log E + Q log Q + (E+Q) × α(n))

- Much faster than answering each query independently

■ Visual Diagram

```
Edges: (0-1:5), (1-2:3), (2-3:7), (0-3:9)
Queries:
  Q1: limit=4 → can use edges with weight <4: only (1-2:3)
  Q2: limit=6 → can use edges weight <6: (1-2:3), (0-1:5)
  Q3: limit=10 → can use all edges

Process:
  Sort queries by limit: Q1(4), Q2(6), Q3(10)
  Pointer i=0 on edges (sorted by weight)

  For Q1: add edges weight<4 → add (1-2)
  For Q2: add edges weight<6 → add (0-1)
  For Q3: add remaining edges → add (2-3), (0-3)
```

■ Template Code

```
public boolean[] offlineQueries(int n, int[][] edges, int[][] queries) {
    // queries: [u, v, limit]
    Arrays.sort(edges, (a,b) -> a[2] - b[2]);  // sort edges by weight

    // Prepare queries with original index
    int q = queries.length;
    int[][] queriesWithIdx = new int[q][4];
    for (int i = 0; i < q; i++) {
        queriesWithIdx[i] = new int[]{queries[i][0], queries[i][1], queries[i][2], i};
    }
    Arrays.sort(queriesWithIdx, (a,b) -> a[2] - b[2]);  // sort by limit

    DSU dsu = new DSU(n);
    boolean[] ans = new boolean[q];
    int edgePtr = 0;

    for (int[] query : queriesWithIdx) {
        int u = query[0], v = query[1], limit = query[2], idx = query[3];

        // Add all edges with weight < limit
        while (edgePtr < edges.length && edges[edgePtr][2] < limit) {
            dsu.union(edges[edgePtr][0], edges[edgePtr][1]);
            edgePtr++;
        }

        ans[idx] = dsu.connected(u, v);
    }
    return ans;
}
```

■ 10 Practice Questions

lem	P
--	--
cking Existence of Edge Length Limited Paths](https://leetcode.com/problems/checking-existence-of-edge-length-limited-paths/)	L
cking Existence of Edge Length Limited Paths II](https://leetcode.com/problems/checking-existence-of-edge-length-limited-paths-ii/)	L
With Minimum Effort](https://leetcode.com/problems/path-with-minimum-effort/)	L
mum Time to Visit a Cell in a Grid](https://leetcode.com/problems/minimum-time-to-visit-a-cell-in-a-grid/)	L
kal's Algorithm for MST](https://www.geeksforgeeks.org/problems/minimum-spanning-tree/1)	G
mum Cost to Connect All Points](https://leetcode.com/problems/minimum-cost-to-connect-all-points/)	L
necting Cities With Minimum Cost](https://leetcode.com/problems/connecting-cities-with-minimum-cost/)	L
ne Dynamic Connectivity](https://www.geeksforgeeks.org/offline-dynamic-connectivity-dsu/)	G
mum Cost to Make at Least One Valid Path](https://www.geeksforgeeks.org/minimum-cost-to-make-at-least-one-valid-path/)	G
ber of Good Paths](https://leetcode.com/problems/number-of-good-paths/)	L

PATTERN 4: String/Email Grouping (Map-Based DSU)

■ Theory

Nodes are **not always integers**. When dealing with strings, emails, or objects:

- **Map each unique value to an index** (0, 1, 2, ...)

- Use DSU on indices

- Track which indices belong together

Properties:

- Requires a `HashMap<String, Integer>` for mapping

- Union entities that share common attributes

■ Visual Diagram

```
Accounts: [
  ["John", "j1@mail.com", "j2@mail.com"],
  ["John", "j2@mail.com", "j3@mail.com"],
  ["Mary", "m1@mail.com"]
]

Step 1: Map emails to indices
  "j1@mail.com" → 0
  "j2@mail.com" → 1
  "j3@mail.com" → 2
  "m1@mail.com" → 3
Step 2: Union emails within same account
```

```
  Account 0: union(0,1)
  Account 1: union(1,2) → now 0,1,2 all connected
  Account 2: 3 alone

Result: Component 1: {0,1,2} (John's emails)
        Component 2: {3} (Mary's email)
```

■ Template Code

```
public List<List<String>> accountsMerge(List<List<String>> accounts) {
    DSU dsu = new DSU(accounts.size());  // One node per account initially
    Map<String, Integer> emailToAccountId = new HashMap<>();

    // First pass: Union accounts that share emails
    for (int i = 0; i < accounts.size(); i++) {
        List<String> account = accounts.get(i);
        for (int j = 1; j < account.size(); j++) {
            String email = account.get(j);
            if (emailToAccountId.containsKey(email)) {
                dsu.union(i, emailToAccountId.get(email));
            } else {
                emailToAccountId.put(email, i);
            }
        }
    }

    // Second pass: Group emails by root account
    Map<Integer, List<String>> rootToEmails = new HashMap<>();
    for (Map.Entry<String, Integer> entry : emailToAccountId.entrySet()) {
        int root = dsu.find(entry.getValue());
        rootToEmails.computeIfAbsent(root, k -> new ArrayList<>()).add(entry.getKey());
    }

    // Format output
    List<List<String>> result = new ArrayList<>();
    for (Map.Entry<Integer, List<String>> entry : rootToEmails.entrySet()) {
        List<String> emails = entry.getValue();
        Collections.sort(emails);
        List<String> merged = new ArrayList<>();
        merged.add(accounts.get(entry.getKey()).get(0));  // name
        merged.addAll(emails);
        result.add(merged);
    }
    return result;
}
```

■ 10 Practice Questions

lem	P
--	--
ounts Merge](https://leetcode.com/problems/accounts-merge/)	L
lar String Groups](https://leetcode.com/problems/similar-string-groups/)	L
tence Similarity II](https://leetcode.com/problems/sentence-similarity-ii/)	L
uate Division](https://leetcode.com/problems/evaluate-division/)	L
All People With Secret](https://leetcode.com/problems/find-all-people-with-secret/)	L

lem	P
up Anagrams Using DSU](https://www.geeksforgeeks.org/group-anagrams-together-using-dsu/)	G
nd Circles in Social Network](https://www.geeksforgeeks.org/friend-circles-using-dsu/)	G
il Account Merging](https://www.geeksforgeeks.org/merging-email-accounts-using-dsu/)	G
If Two Strings are Equivalent](https://www.geeksforgeeks.org/check-if-two-strings-are-equivalent-using-dsu/)	G
cographically Smallest String After Swaps](https://www.geeksforgeeks.org/lexicographically-smallest-string-after-swaps-using-dsu/)	G

PATTERN 5: Lexicographical/String Ordering After Swaps

■ Theory

You have positions that can be swapped freely. The positions that are **connected** (directly or indirectly) can be rearranged arbitrarily.

Properties:

- Union positions that are connected by swap operations

- For each component, collect characters at those positions

- Sort characters in descending order

- Place back in ascending order of positions

■ Visual Diagram

```
String: "dcab"
Swaps: [0,3], [1,2]

Step 1: Build connections
  Union(0,3)
  Union(1,2)

Step 2: Components
  Component A: positions {0,3} → characters {'d','b'}
  Component B: positions {1,2} → characters {'c','a'}

Step 3: Sort each component's characters
  Component A sorted: {'b','d'}
  Component B sorted: {'a','c'}

Step 4: Place back (positions sorted ascending)
  pos0 (A) → 'b'
  pos1 (B) → 'a'
  pos2 (B) → 'c'
  pos3 (A) → 'd'

Result: "bacd"
```

■ Template Code

```
public String smallestStringWithSwaps(String s, List<List<Integer>> pairs) {
    int n = s.length();
    DSU dsu = new DSU(n);

    // Union all positions that can be swapped
    for (List<Integer> pair : pairs) {
        dsu.union(pair.get(0), pair.get(1));
    }

    // Group indices by root
    Map<Integer, List<Integer>> rootToIndices = new HashMap<>();
    for (int i = 0; i < n; i++) {
        int root = dsu.find(i);
        rootToIndices.computeIfAbsent(root, k -> new ArrayList<>()).add(i);
    }

    // Build result
    char[] result = new char[n];
    for (Map.Entry<Integer, List<Integer>> entry : rootToIndices.entrySet()) {
        List<Integer> indices = entry.getValue();

        // Get characters at these indices
        List<Character> chars = new ArrayList<>();
        for (int idx : indices) {
            chars.add(s.charAt(idx));
        }

        // Sort characters ascending
        Collections.sort(chars);
        // Sort indices ascending
        Collections.sort(indices);

        // Place smallest char at smallest index
        for (int i = 0; i < indices.size(); i++) {
            result[indices.get(i)] = chars.get(i);
        }
    }
    return new String(result);
}
```

■ 10 Practice Questions

roblem	Platf
-------	-------
Smallest String With Swaps](https://leetcode.com/problems/smallest-string-with-swaps/)	Leet
Smallest String After Swaps](https://www.geeksforgeeks.org/smallest-string-after-swaps-using-dsu/)	GFG
Minimum Swaps to Make String Equal](https://www.geeksforgeeks.org/minimum-swaps-to-make-string-equal-using-dsu/)	GFG
Lexicographically Smallest Equivalent String](https://leetcode.com/problems/lexicographically-smallest-equivalent-string/)	Leet
Smallest Equivalent String After Swaps](https://www.geeksforgeeks.org/smallest-equivalent-string-after-swaps/)	GFG
Rearrange String k Distance Apart](https://www.geeksforgeeks.org/rearrange-string-k-distance-apart-using-dsu/)	GFG
String Transformation Using DSU](https://www.geeksforgeeks.org/string-transformation-using-dsu/)	GFG

roblem	Platf
Make String Sorted by Swapping](https://www.geeksforgeeks.org/make-string-sorted-by-swapping-using-dsu/)	GFG
Minimum Swaps to Group All 1's Together](https://www.geeksforgeeks.org/minimum-swaps-to-group-all-1s-together-using-dsu/)	GFG
Sort String by Swapping with Restrictions](https://www.geeksforgeeks.org/sort-string-by-swapping-with-restrictions-using-dsu/)	GFG

PATTERN 6: Two-Player/Two-Entity Connectivity

■ Theory

Problems with **two entities** (Alice and Bob, two networks, etc.). You maintain **two separate DSU structures** or track connectivity for both simultaneously.

Properties:

- Process shared edges first (they benefit both)

- Then process entity-specific edges

- Track component counts for each entity separately

■ Visual Diagram

```
Edges:
  Type 1: Alice-only
  Type 2: Bob-only
  Type 3: Both can use

Strategy:
  1. Use Type 3 edges first for BOTH DSUs
  2. Then use Type 1 for Alice's DSU
  3. Then use Type 2 for Bob's DSU
  4. Count unused edges = total - used
```

■ Template Code

```
public int maxNumEdgesToRemove(int n, int[][] edges) {
    DSU alice = new DSU(n);
    DSU bob = new DSU(n);
    int edgesUsed = 0;

    // First: Process type 3 edges (both can use)
    for (int[] edge : edges) {
        if (edge[0] == 3) {
            if (alice.union(edge[1]-1, edge[2]-1) | bob.union(edge[1]-1, edge[2]-1)) {
                edgesUsed++;
            }
        }
    }
```

```
        // Second: Process type 1 edges (Alice only)
        for (int[] edge : edges) {
            if (edge[0] == 1) {
                if (alice.union(edge[1]-1, edge[2]-1)) {
                    edgesUsed++;
                }
            }
        }

        // Third: Process type 2 edges (Bob only)
        for (int[] edge : edges) {
            if (edge[0] == 2) {
                if (bob.union(edge[1]-1, edge[2]-1)) {
                    edgesUsed++;
                }
            }
        }

        // Check if both are fully connected
        if (alice.countComponents() != 1 || bob.countComponents() != 1) {
            return -1;
        }

        return edges.length - edgesUsed;
    }
```

■ 10 Practice Questions

lumber of Edges to Keep Graph Fully Traversable](https://leetcode.com/problems/remove-max-number-of-edges-to-keep-graph-fully-tra
etworks are Connected](https://www.geeksforgeeks.org/check-if-two-networks-are-connected-using-dsu/)
s to Add to Make Two Graphs Connected](https://www.geeksforgeeks.org/minimum-edges-to-add-to-make-two-graphs-connected-using
ys to Reconnect Two Networks](https://www.geeksforgeeks.org/number-of-ways-to-reconnect-two-networks-using-dsu/)
th Two Types of Edges](https://www.geeksforgeeks.org/connectivity-with-two-types-of-edges-using-dsu/)
ivity with Two Players](https://www.geeksforgeeks.org/graph-connectivity-with-two-players-using-dsu/)
to Connect Two Groups](https://www.geeksforgeeks.org/minimum-cost-to-connect-two-groups-using-dsu/)
raphs are Isomorphic](https://www.geeksforgeeks.org/check-if-two-graphs-are-isomorphic-using-dsu/)
aphs Using DSU](https://www.geeksforgeeks.org/merge-two-graphs-using-dsu/)
ations to Make Two Graphs Identical](https://www.geeksforgeeks.org/minimum-operations-to-make-two-graphs-identical-using-dsu/)

PATTERN 7: Prime Factor / GCD-Based Union

■ Theory

Elements are connected if they **share a common factor > 1**. Instead of checking all pairs (O(n²)), union each number with its **prime factors**.

Properties:

- For each number, find its prime factors

- Union the number with each factor (treat factors as nodes)

- After processing, numbers with shared factors will be in same component

■ Visual Diagram

```
Numbers: [2, 3, 6, 7, 14]

Step 1: Map numbers to indices
  2→0, 3→1, 6→2, 7→3, 14→4

Step 2: For each number, union with prime factors
  2: factors [2] → union(0, 2)
  3: factors [3] → union(1, 3)
  6: factors [2,3] → union(2,2), union(2,3)
  7: factors [7] → union(3,7)
  14: factors [2,7] → union(4,2), union(4,7)

Result:
  Component A: {2,3,6,7,14} (all connected via 2 and 7)
  Wait, 3 is connected? Yes via 6 (3 and 6 share factor 3)
  Actually 3 connects to 6, 6 connects to 2, 2 connects to 14, 14 connects to 7
  So ALL numbers are connected!
```

■ Template Code

```
public int largestComponentSize(int[] nums) {
    int n = nums.length;
    DSU dsu = new DSU(n);
    Map<Integer, List<Integer>> factorToNumbers = new HashMap<>();

    // For each number, find its factors
    for (int i = 0; i < n; i++) {
        int num = nums[i];
        int temp = num;
        for (int f = 2; f * f <= temp; f++) {
            if (temp % f == 0) {
                while (temp % f == 0) temp /= f;
                factorToNumbers.computeIfAbsent(f, k -> new ArrayList<>()).add(i);
            }
        }
        if (temp > 1) {
            factorToNumbers.computeIfAbsent(temp, k -> new ArrayList<>()).add(i);
        }
    }

    // Union numbers that share factors
    for (List<Integer> numberIndices : factorToNumbers.values()) {
        int first = numberIndices.get(0);
        for (int i = 1; i < numberIndices.size(); i++) {
            dsu.union(first, numberIndices.get(i));
```

```
                }
            }

            // Count largest component size
            int[] size = new int[n];
            int maxSize = 0;
            for (int i = 0; i < n; i++) {
                int root = dsu.find(i);
                size[root]++;
                maxSize = Math.max(maxSize, size[root]);
            }
            return maxSize;
        }
```

■ 10 Practice Questions

onent Size by Common Factor](https://leetcode.com/problems/largest-component-size-by-common-factor/)
mon Divisor Traversal](https://leetcode.com/problems/greatest-common-divisor-traversal/)
nnected Components in GCD Graph](https://www.geeksforgeeks.org/number-of-connected-components-in-gcd-graph-using-dsu/)
rs by GCD](https://www.geeksforgeeks.org/group-numbers-by-gcd-using-dsu/)
D After Operations](https://www.geeksforgeeks.org/maximum-gcd-after-operations-using-dsu/)
an Be Divided into Pairs with GCD > 1](https://www.geeksforgeeks.org/find-if-array-can-be-divided-into-pairs-with-gcd-1-using-dsu/)
to Make Array GCD 1](https://www.geeksforgeeks.org/minimum-cost-to-make-array-gcd-1-using-dsu/)
bers Using Prime Factors](https://www.geeksforgeeks.org/connect-numbers-using-prime-factors-using-dsu/)
e of Connected Component in GCD Graph](https://www.geeksforgeeks.org/maximum-size-of-connected-component-in-gcd-graph-using
ith GCD > 1 Using DSU](https://www.geeksforgeeks.org/count-pairs-with-gcd-1-using-dsu/)

PATTERN 8: Equality/Inequality Equations (Satisfiability)

■ Theory

Given equations like `"a==b"`, `"c!=d"`, determine if all can be satisfied simultaneously.

Properties:

- **First pass**: Union all variables connected by `==` equations

- **Second pass**: Check all `!=` equations — if both variables in same component → contradiction

■ Visual Diagram

```
Equations:
  a==b, b==c, c!=d, d==e

Step 1: Process == equations
  Union(a,b), Union(b,c) → {a,b,c} in same component
  Union(d,e) → {d,e} in same component

Step 2: Process != equations
  Check c!=d: c in {a,b,c}, d in {d,e} → Different components ■ OK

If we had a!=b:
  a and b in same component → Contradiction! ■
```

■ Template Code

```
public boolean equationsPossible(String[] equations) {
    DSU dsu = new DSU(26);  // 26 lowercase letters

    // First pass: Union all '==' equations
    for (String eq : equations) {
        if (eq.charAt(1) == '=') {
            int x = eq.charAt(0) - 'a';
            int y = eq.charAt(3) - 'a';
            dsu.union(x, y);
        }
    }

    // Second pass: Check all '!=' equations
    for (String eq : equations) {
        if (eq.charAt(1) == '!') {
            int x = eq.charAt(0) - 'a';
            int y = eq.charAt(3) - 'a';
            if (dsu.find(x) == dsu.find(y)) {
                return false;  // Contradiction
            }
        }
    }
    return true;
}
```

■ 10 Practice Questions

ality Equations](https://leetcode.com/problems/satisfiability-of-equality-equations/)
ions](https://www.geeksforgeeks.org/possibility-of-equations-using-dsu/)
are Consistent](https://www.geeksforgeeks.org/check-if-equations-are-consistent-using-dsu/)
Expressions](https://www.geeksforgeeks.org/evaluate-boolean-expressions-using-dsu/)
ueries](https://www.geeksforgeeks.org/variable-equality-queries-using-dsu/)
re Satisfiable](https://www.geeksforgeeks.org/find-if-equations-are-satisfiable-using-dsu/)
of Changes to Make Equations Satisfiable](https://www.geeksforgeeks.org/minimum-number-of-changes-to-make-equations-satisfiable-

ing DSU](https://www.geeksforgeeks.org/equation-solver-using-dsu/)
s are Consistent](https://www.geeksforgeeks.org/check-if-constraints-are-consistent-using-dsu/)
Evaluation](https://www.geeksforgeeks.org/logical-expression-evaluation-using-dsu/)

PATTERN 9: Minimum Time to Connect / Binary Search + DSU

■ Theory

Problems asking "minimum maximum effort" or "earliest time when nodes become connected" — use **binary search on answer** + DSU to check feasibility.

Properties:

- Binary search on the threshold (time, effort, distance)

- For a given threshold, union all edges with weight ≤ threshold

- Check if source and target are connected

■ Visual Diagram

```
Find minimum effort path from top-left to bottom-right:

Grid:
[0, 2, 5]
[3, 8, 1]
[4, 6, 7]

Binary Search on effort:
  low=0, high=8
  mid=4: Can we reach with max effort ≤4?
    Union edges with diff ≤4
    Check if (0,0) and (2,2) connected? No → low=5
  mid=6: Can we reach with max effort ≤6?
    Union edges with diff ≤6
    Check if connected? Yes → high=6
  Result: 6
```

■ Template Code

```
public int minimumEffortPath(int[][] heights) {
    int m = heights.length, n = heights[0].length;
    int left = 0, right = 1000000;
    int ans = right;

    while (left <= right) {
        int mid = left + (right - left) / 2;
```

```
            if (canReach(heights, mid)) {
                ans = mid;
                right = mid - 1;
            } else {
                left = mid + 1;
            }
        }
        return ans;
    }

    private boolean canReach(int[][] heights, int maxDiff) {
        int m = heights.length, n = heights[0].length;
        DSU dsu = new DSU(m * n);
        int[][] dirs = {{0,1},{1,0},{0,-1},{-1,0}};

        for (int i = 0; i < m; i++) {
            for (int j = 0; j < n; j++) {
                int idx = i * n + j;
                for (int[] d : dirs) {
                    int ni = i + d[0], nj = j + d[1];
                    if (ni >= 0 && ni < m && nj >= 0 && nj < n) {
                        int nidx = ni * n + nj;
                        if (Math.abs(heights[i][j] - heights[ni][nj]) <= maxDiff) {
                            dsu.union(idx, nidx);
                        }
                    }
                }
            }
        }

        return dsu.connected(0, m * n - 1);
    }
```

■ 10 Practice Questions

em
--
With Minimum Effort](https://leetcode.com/problems/path-with-minimum-effort/)
in Rising Water](https://leetcode.com/problems/swim-in-rising-water/)
mum Time to Visit a Cell in a Grid](https://leetcode.com/problems/minimum-time-to-visit-a-cell-in-a-grid/)
mum Cost to Make at Least One Valid Path](https://leetcode.com/problems/minimum-cost-to-make-at-least-one-valid-path-in-a-grid/)
est Time When Nodes Become Connected](https://www.geeksforgeeks.org/earliest-time-when-nodes-become-connected-using-dsu/)
mum Maximum Distance in a Graph](https://www.geeksforgeeks.org/minimum-maximum-distance-in-a-graph-using-dsu/)
nect Cities with Minimum Cost](https://www.geeksforgeeks.org/connect-cities-with-minimum-cost-using-dsu/)
mum Time to Reach Destination](https://www.geeksforgeeks.org/minimum-time-to-reach-destination-using-dsu/)
with Minimum Maximum Edge Weight](https://www.geeksforgeeks.org/path-with-minimum-maximum-edge-weight-using-dsu/)
mize the Maximum Difference in a Path](https://www.geeksforgeeks.org/minimize-the-maximum-difference-in-a-path-using-dsu/)

PATTERN 10: Reverse Thinking / Time-Travel DSU

■ Theory

Problems that ask "what happens if we remove edges" — instead of removing, **add edges in reverse order**.

Properties:

- Start from the final state (after all removals)

- Process operations backwards

- When we "remove" in original, we "add" in reverse

■ Visual Diagram

```
Original: Remove edges in order: e1, e2, e3
          0---1---2
          |       |
          3-------4

Forward thinking (hard):
  Start with graph, remove e1, track connectivity changes

Reverse thinking (easy):
  Start with graph MINUS e1,e2,e3
  Add e3, then e2, then e1
  Track when components merge
```

■ Template Code

```
public int[] findRedundantConnection(int[][] edges) {
    int n = edges.length;
    DSU dsu = new DSU(n + 1);

    for (int[] edge : edges) {
        if (!dsu.union(edge[0], edge[1])) {
            return edge;  // This edge creates a cycle
        }
    }
    return new int[0];
}

// For removal problems:
public List<Boolean> areConnectedAfterRemovals(int n, int[][] edges, int[][] queries) {
    // Process queries in reverse: removals become additions
    DSU dsu = new DSU(n);
    Set<String> removed = new HashSet<>();

    // Mark edges that will be removed
    for (int[] q : queries) {
        removed.add(q[0] + "," + q[1]);
    }

    // Add edges that are never removed
```

```
        for (int[] e : edges) {
            if (!removed.contains(e[0] + "," + e[1])) {
                dsu.union(e[0], e[1]);
            }
        }

        // Process queries in reverse
        List<Boolean> ans = new ArrayList<>();
        for (int i = queries.length - 1; i >= 0; i--) {
            ans.add(dsu.connected(queries[i][0], queries[i][1]));
            dsu.union(queries[i][0], queries[i][1]);  // Add edge back
        }
        Collections.reverse(ans);
        return ans;
    }
```

■ 10 Practice Questions

nt Connection](https://leetcode.com/problems/redundant-connection/)
Existence of Edge Length Limited Paths](https://leetcode.com/problems/checking-existence-of-edge-length-limited-paths/)
Connectivity Queries](https://www.geeksforgeeks.org/dynamic-connectivity-queries-using-dsu/)
ynamic Connectivity](https://www.geeksforgeeks.org/offline-dynamic-connectivity-dsu/)
n Nodes Become Disconnected](https://www.geeksforgeeks.org/find-when-nodes-become-disconnected-using-dsu/)
Edges to Remove to Disconnect Graph](https://www.geeksforgeeks.org/minimum-edges-to-remove-to-disconnect-graph-using-dsu/)
Queries of Edge Removals](https://www.geeksforgeeks.org/process-queries-of-edge-removals-using-dsu/)
vity After Edge Removals](https://www.geeksforgeeks.org/connectivity-after-edge-removals-using-dsu/)
f Connected Components After Removals](https://www.geeksforgeeks.org/number-of-connected-components-after-removals-using-dsu
Graph Connectivity Queries](https://www.geeksforgeeks.org/dynamic-graph-connectivity-queries-using-dsu/)

SUMMARY: DSU Pattern Recognition Table

ne	Problem Signal	Key Insight	Time Cor
	---	---	---
nectivity**	"Find connected components", "Merge groups"	Count roots after all unions	O(E × α(n
ction**	"Find redundant edge", "Is this a tree?"	If find(u)==find(v) before union → cycle	O(E × α(n
ries**	"Answer queries with thresholds", "Process in order"	Sort queries + edges, process incrementally	O((E+Q) l
il Grouping**	"Merge accounts", "Group similar strings"	Map strings to indices first	O(N log N
hical Ordering**	"Smallest string after swaps"	Sort each component's characters independently	O(N log N
Connectivity**	"Alice and Bob can use different edges"	Process shared edges first	O(E × α(n

ne	Problem Signal	Key Insight	Time Cor
or Union**	"Connected by common factor > 1"	Union with prime factors, not numbers	O(N × √m
equality**	"a==b, c!=d equations"	Union '==', check '!='	O(N × α(2
rch + DSU**	"Minimize maximum effort", "Earliest time"	Binary search + DSU check	O(log(max
inking**	"Removing edges", "Time-travel queries"	Process operations backwards	O((E+Q) ×

Quick Reference: When to Use Which Pattern

```
"Count components" → Pattern 1 (Basic Connectivity)
"Find cycle/extra edge" → Pattern 2 (Cycle Detection)
"Queries with thresholds" → Pattern 3 (Offline Queries)
"Merge accounts/emails" → Pattern 4 (String Grouping)
"Swap to get smallest string" → Pattern 5 (Lexicographical)
"Alice and Bob" → Pattern 6 (Two-Player)
"Share common factor" → Pattern 7 (Prime Factor)
"a==b, c!=d equations" → Pattern 8 (Equality)
"Minimize maximum effort" → Pattern 9 (Binary Search + DSU)
"Removing edges dynamically" → Pattern 10 (Reverse Thinking)
```

■ **Graph Algorithms Handbook (Pattern-Based Edition)**

Part 2: Dijkstra's Algorithm

DIJKSTRA'S ALGORITHM - Complete Pattern Guide

■ **Properties & Complexity**

Property	Value
----------	-------
Time Complexity	O((V+E) log V) with binary heap
Space Complexity	O(V)

Property	Value
Works On	Weighted graphs with **NON-NEGATIVE** weights
Finds	Shortest path from single source to all nodes
Greedy Property	Once node is popped from PQ, its distance is final

When to Use Dijkstra (The Signals)

Problem Signal	Pattern
----------	-------
"Shortest path in weighted graph"	Pattern 1
"Minimum time/cost to reach destination"	Pattern 1
"Network delay time"	Pattern 2
"Path with minimum maximum edge weight"	Pattern 3
"Cheapest flight with at most K stops"	Pattern 4
"Shortest path with obstacles/teleporters"	Pattern 5
"Minimum cost to make at least one valid path"	Pattern 6
"Shortest path in matrix with weights"	Pattern 7
"Path with minimum effort"	Pattern 8
"Reachable nodes within cost limit"	Pattern 9

PATTERN 1: Basic Dijkstra (Single Source Shortest Path)

■ Theory

The **fundamental Dijkstra pattern**. Start from source node, use priority queue to always expand the node with smallest known distance. For each neighbor, if new distance is smaller, update and push to PQ.

Properties:

- Once a node is popped from PQ, its distance is finalized (greedy choice property)

- Works only with non-negative weights

- If graph is unweighted, BFS is faster (O(V+E) vs O((V+E)log V))

■ Visual Diagram

```
Graph:     0 --4--> 1
           |        |
           2        1
           |        |
           2 --3--> 3

Dijkstra from node 0:

Initial: dist = [0, ∞, ∞, ∞], PQ = [(0,0)]

Step 1: Pop (0,0)
  Neighbors: 1 (4), 2 (2)
  dist[1] = 4, dist[2] = 2
  PQ = [(2,2), (4,1)]

Step 2: Pop (2,2)
  Neighbors: 3 (3)
  dist[3] = 5
  PQ = [(4,1), (5,3)]

Step 3: Pop (4,1)
  Neighbors: 3 (1)
  dist[3] = min(5, 4+1=5) → 5
  PQ = [(5,3)]

Result: dist = [0, 4, 2, 5]
```

■ Template Code

```
public int[] dijkstra(int n, List<int[]>[] graph, int src) {
    int[] dist = new int[n];
    Arrays.fill(dist, Integer.MAX_VALUE);
    dist[src] = 0;

    // Priority queue: [distance, node]
    PriorityQueue<int[]> pq = new PriorityQueue<>((a,b) -> a[0] - b[0]);
    pq.offer(new int[]{0, src});

    while (!pq.isEmpty()) {
        int[] curr = pq.poll();
        int d = curr[0], u = curr[1];

        // Skip stale entries
        if (d > dist[u]) continue;

        for (int[] edge : graph[u]) {
            int v = edge[0], w = edge[1];
            if (dist[u] + w < dist[v]) {
                dist[v] = dist[u] + w;
                pq.offer(new int[]{dist[v], v});
            }
        }
    }
    return dist;
}
```

■ 10 Practice Questions

elay Time](https://leetcode.com/problems/network-delay-time/)
ath in Weighted Graph](https://www.geeksforgeeks.org/problems/shortest-path-in-weighted-graph/1)
Minimum Effort](https://leetcode.com/problems/path-with-minimum-effort/)
Cost Path in Grid](https://www.geeksforgeeks.org/problems/minimum-cost-path/1)
ity With Smallest Neighbors](https://leetcode.com/problems/find-the-city-with-the-smallest-number-of-neighbors-at-a-threshold-distance
gorithm](https://www.geeksforgeeks.org/problems/implementing-dijkstra-set-1-adjacency-matrix/1)
Time to Visit a Cell](https://www.geeksforgeeks.org/minimum-time-to-visit-a-cell-in-a-grid/)
ath in Binary Matrix](https://leetcode.com/problems/shortest-path-in-binary-matrix/)
e Nodes in Subdivided Graph](https://leetcode.com/problems/reachable-nodes-in-subdivided-graph/)
istance in a Maze](https://www.geeksforgeeks.org/shortest-distance-in-a-maze/)

PATTERN 2: Multiple Sources / Multi-Source Dijkstra

■ Theory

When you have **multiple starting points** and need the shortest distance to all nodes from **any source**. Initialize PQ with ALL sources at distance 0.

Properties:

- Same as single-source, but PQ starts with all sources

- Distances represent distance to nearest source

- Perfect for "nearest hospital", "nearest fire station" problems

■ Visual Diagram

```
Grid with sources (0 = source, ∞ = empty):
[0] [∞] [∞]
[∞] [0] [∞]
[∞] [∞] [∞]

Multi-source Dijkstra:
Initialize PQ with both (0,0,0) and (0,1,1)
Propagate outward like BFS but with weights

Result distances:
[0] [1] [2]
[1] [0] [1]
[2] [1] [2]
```

■ Template Code

```
public int[] multiSourceDijkstra(int n, List<int[]>[] graph, int[] sources) {
    int[] dist = new int[n];
    Arrays.fill(dist, Integer.MAX_VALUE);
    PriorityQueue<int[]> pq = new PriorityQueue<>((a,b) -> a[0] - b[0]);

    // Add all sources with distance 0
    for (int src : sources) {
        dist[src] = 0;
        pq.offer(new int[]{0, src});
    }

    while (!pq.isEmpty()) {
        int[] curr = pq.poll();
        int d = curr[0], u = curr[1];
        if (d > dist[u]) continue;

        for (int[] edge : graph[u]) {
            int v = edge[0], w = edge[1];
            if (dist[u] + w < dist[v]) {
                dist[v] = dist[u] + w;
                pq.offer(new int[]{dist[v], v});
            }
        }
    }
    return dist;
}
```

■ 10 Practice Questions

m
um Time to Visit All Apples](https://www.geeksforgeeks.org/minimum-time-to-visit-all-apples/)
st Distance from All Buildings](https://leetcode.com/problems/shortest-distance-from-all-buildings/)
um Cost to Reach Destination with Multiple Starts](https://www.geeksforgeeks.org/minimum-cost-to-reach-destination-with-multiple-star
t Distance to Any Source](https://www.geeksforgeeks.org/closest-distance-to-any-source-using-dijkstra/)
earest Node with Given Value](https://www.geeksforgeeks.org/find-nearest-node-with-given-value-using-dijkstra/)
st Path to Get All Keys](https://leetcode.com/problems/shortest-path-to-get-all-keys/)
um Number of Refueling Stops](https://leetcode.com/problems/minimum-number-of-refueling-stops/)
e the Spreading Fire](https://leetcode.com/problems/escape-the-spreading-fire/)
st Exit from Entrance in Maze](https://leetcode.com/problems/nearest-exit-from-entrance-in-maze/)
um Time to Visit All Points](https://www.geeksforgeeks.org/minimum-time-to-visit-all-points-using-dijkstra/)

PATTERN 3: Minimize Maximum Edge Weight (Minimax Path)

■ Theory

Find a path from source to destination that **minimizes the maximum edge weight** along the path. Use **modified Dijkstra** where the "distance" stored is the maximum edge weight so far.

Properties:

- Relaxation: `newMax = max(currentMax, edgeWeight)`

- If `newMax < dist[v]`, update

- Can also be solved with Binary Search + BFS/DSU

■ Visual Diagram

```
Path options from 0 to 3:

Path 1: 0→1 (5) → 3 (2) → max = max(5,2) = 5
Path 2: 0→2 (3) → 3 (4) → max = max(3,4) = 4
Path 3: 0→1 (5) → 2 (1) → 3 (4) → max = max(5,1,4) = 5

Minimax path: Path 2 with max edge = 4
```

■ Template Code

```
public int minimaxPath(int n, List<int[]>[] graph, int src, int dest) {
    int[] maxEdge = new int[n];
    Arrays.fill(maxEdge, Integer.MAX_VALUE);
    maxEdge[src] = 0;

    PriorityQueue<int[]> pq = new PriorityQueue<>((a,b) -> a[0] - b[0]);
    pq.offer(new int[]{0, src});

    while (!pq.isEmpty()) {
        int[] curr = pq.poll();
        int maxSoFar = curr[0], u = curr[1];

        if (u == dest) return maxSoFar;
        if (maxSoFar > maxEdge[u]) continue;

        for (int[] edge : graph[u]) {
            int v = edge[0], w = edge[1];
            int newMax = Math.max(maxSoFar, w);
            if (newMax < maxEdge[v]) {
                maxEdge[v] = newMax;
                pq.offer(new int[]{newMax, v});
            }
        }
    }
    return -1;
}
```

■ 10 Practice Questions

lem	
--	
With Minimum Effort](https://leetcode.com/problems/path-with-minimum-effort/)	
n in Rising Water](https://leetcode.com/problems/swim-in-rising-water/)	
mum Maximum Distance in a Graph](https://www.geeksforgeeks.org/minimum-maximum-distance-in-a-graph-using-dijkstra/)	
mize the Maximum Edge Weight](https://www.geeksforgeeks.org/minimize-the-maximum-edge-weight-using-dijkstra/)	
with Minimum Maximum Value](https://www.geeksforgeeks.org/path-with-minimum-maximum-value-using-dijkstra/)	
mum Time to Visit a Cell](https://leetcode.com/problems/minimum-time-to-visit-a-cell-in-a-grid/)	
mum Cost to Make at Least One Valid Path](https://leetcode.com/problems/minimum-cost-to-make-at-least-one-valid-path-in-a-grid/)	
ch End with Minimum Maximum Jump](https://www.geeksforgeeks.org/reach-end-with-minimum-maximum-jump-using-dijkstra/)	
mize Maximum Distance to Gas Station](https://www.geeksforgeeks.org/minimize-maximum-distance-to-gas-station-using-dijkstra/)	
mum Obstacle Removal to Reach Corner](https://leetcode.com/problems/minimum-obstacle-removal-to-reach-corner/)	

PATTERN 4: K Stops / Limited Hops (Modified Dijkstra)

■ Theory

Find shortest path with **constraint on number of edges/hops/stops**. Use `dist[node][k]` where k is stops used. Or use BFS + relaxation with stop count.

Properties:

- Bellman-Ford is more natural for K stops, but Dijkstra works with `(node, stops)` state

- Stop constraint makes graph "layered" by number of hops used

■ Visual Diagram

```
Flights: 0→1 (100), 0→2 (500), 1→2 (100), 1→3 (500), 2→3 (100)
K=1 stop (max 2 edges)

From 0 to 3 with K=1:
  0→1→3: cost = 100 + 500 = 600
  0→2→3: cost = 500 + 100 = 600
  0→1→2→3: NOT allowed (2 stops, K=1)

Dijkstra with state (node, stops):
  Start: (0,0) cost=0
  (0,0) → (1,1) cost=100, (2,1) cost=500
  (1,1) → (3,2) cost=600 (but stops=2 > K? No, K=1 means stops≤1)
  Wait, stops=2 means used 2 flights = 1 stop? Let's clarify:
    Stops = intermediate airports = edges - 1
    K=1 stop means max 2 edges
    (node, edgesUsed) is clearer
```

■ Template Code

```
public int findCheapestPrice(int n, int[][] flights, int src, int dst, int k) {
    List<int[]>[] graph = new ArrayList[n];
    for (int i = 0; i < n; i++) graph[i] = new ArrayList<>();
    for (int[] f : flights) graph[f[0]].add(new int[]{f[1], f[2]});

    int[] dist = new int[n];
    Arrays.fill(dist, Integer.MAX_VALUE);
    dist[src] = 0;

    // Priority queue: [cost, node, stops]
    PriorityQueue<int[]> pq = new PriorityQueue<>((a,b) -> a[0] - b[0]);
    pq.offer(new int[]{0, src, 0});

    while (!pq.isEmpty()) {
        int[] curr = pq.poll();
        int cost = curr[0], u = curr[1], stops = curr[2];

        if (u == dst) return cost;
        if (stops > k) continue;

        for (int[] edge : graph[u]) {
            int v = edge[0], w = edge[1];
            if (cost + w < dist[v]) {
                dist[v] = cost + w;
                pq.offer(new int[]{cost + w, v, stops + 1});
            }
        }
    }
    return -1;
}
```

■ 10 Practice Questions

Flights Within K Stops](https://leetcode.com/problems/cheapest-flights-within-k-stops/)
Cost to Reach Destination with K Stops](https://www.geeksforgeeks.org/minimum-cost-to-reach-destination-with-k-stops/)
ath with Exactly K Edges](https://www.geeksforgeeks.org/shortest-path-with-exactly-k-edges/)
Ways to Reach Destination with K Stops](https://www.geeksforgeeks.org/number-of-ways-to-reach-destination-with-k-stops/)
ime to Reach Destination with K Stops](https://www.geeksforgeeks.org/minimum-time-to-reach-destination-with-k-stops/)
ath with Maximum K Edges](https://www.geeksforgeeks.org/shortest-path-with-maximum-k-edges-using-dijkstra/)
Flights with K Stops (GFG Version)](https://www.geeksforgeeks.org/cheapest-flights-within-k-stops/)
Cost to Reach End with at Most K Jumps](https://www.geeksforgeeks.org/minimum-cost-to-reach-end-with-at-most-k-jumps-using-dijkstr
st Path with K Stops](https://www.geeksforgeeks.org/kth-smallest-path-with-k-stops-using-dijkstra/)
uel Cost to Reach Destination with K Stops](https://www.geeksforgeeks.org/minimum-fuel-cost-to-reach-destination-with-k-stops/)

PATTERN 5: Dijkstra on Grid with Weights

■ Theory

When moving in a grid where each cell has a cost, or moving in 4/8 directions has different weights. Treat each cell as a node, neighbors are adjacent cells.

Properties:

- Grid becomes a graph with V = rows × cols

- Each cell connects to up to 4 neighbors

- Weight can be cell value or movement cost

■ Visual Diagram

```
Grid with cell costs:
[1, 2, 3]
[4, 5, 6]
[7, 8, 9]

From (0,0) to (2,2):
Path: (0,0)[1] → (0,1)[2] → (1,1)[5] → (2,2)[9]
Total cost = 1+2+5+9 = 17

Dijkstra on grid:
  dist[0][0] = 1
  From (0,0): (0,1) cost=1+2=3, (1,0) cost=1+4=5
  ...
```

■ Template Code

```
public int minCostPath(int[][] grid) {
    int m = grid.length, n = grid[0].length;
    int[][] dist = new int[m][n];
    for (int[] row : dist) Arrays.fill(row, Integer.MAX_VALUE);
    dist[0][0] = grid[0][0];

    PriorityQueue<int[]> pq = new PriorityQueue<>((a,b) -> a[2] - b[2]);
    pq.offer(new int[]{0, 0, grid[0][0]});

    int[][] dirs = {{0,1},{1,0},{0,-1},{-1,0}};

    while (!pq.isEmpty()) {
        int[] curr = pq.poll();
        int i = curr[0], j = curr[1], cost = curr[2];

        if (i == m-1 && j == n-1) return cost;
        if (cost > dist[i][j]) continue;

        for (int[] d : dirs) {
            int ni = i + d[0], nj = j + d[1];
            if (ni >= 0 && ni < m && nj >= 0 && nj < n) {
```

```
                    int newCost = cost + grid[ni][nj];
                    if (newCost < dist[ni][nj]) {
                        dist[ni][nj] = newCost;
                        pq.offer(new int[]{ni, nj, newCost});
                    }
                }
            }
        }
        return -1;
    }
```

■ 10 Practice Questions

em
-
num Cost Path in Grid](https://www.geeksforgeeks.org/problems/minimum-cost-path/1)
num Path Cost in a Grid](https://leetcode.com/problems/minimum-path-cost-in-a-grid/)
num Cost Homecoming of a Robot](https://leetcode.com/problems/minimum-cost-homecoming-of-a-robot-in-a-grid/)
num Cost to Make at Least One Valid Path](https://leetcode.com/problems/minimum-cost-to-make-at-least-one-valid-path-in-a-grid/)
est Path in Grid with Obstacles](https://www.geeksforgeeks.org/shortest-path-in-grid-with-obstacles-using-dijkstra/)
num Cost to Reach Destination in Grid](https://www.geeksforgeeks.org/minimum-cost-to-reach-destination-in-grid-using-dijkstra/)
num Time to Visit a Cell in a Grid](https://leetcode.com/problems/minimum-time-to-visit-a-cell-in-a-grid/)
num Moves to Reach Target with Rotations](https://leetcode.com/problems/minimum-moves-to-reach-target-with-rotations/)
num Cost Path with Diagonal Moves](https://www.geeksforgeeks.org/minimum-cost-path-with-diagonal-moves-using-dijkstra/)
num Cost to Reach End with Teleporters](https://www.geeksforgeeks.org/minimum-cost-to-reach-end-with-teleporters-using-dijkstra/)

PATTERN 6: Dijkstra with State (Bitmask + Position)

■ Theory

When the graph has **state** (keys collected, visited nodes, etc.). State is represented as bitmask. Node becomes `(position, state)`. Transitions modify state.

Properties:

- Total states = positions × 2^k where k = number of keys/items

- Dijkstra works on state graph

- Used in "shortest path to collect all keys" problems

■ Visual Diagram

```
Grid with keys (a,b) and doors (A,B):
Start at (0,0) with mask=00

State space:
(0,0,00) → pick key 'a' → (0,0,01)
(0,0,01) → can open door 'A'
(0,0,01) → pick key 'b' → (0,0,11)

Each cell has up to 2^k states
```

■ Template Code

```
public int shortestPathToGetAllKeys(String[] grid) {
    int m = grid.length, n = grid[0].length();
    int totalKeys = 0;
    int startX = 0, startY = 0;

    for (int i = 0; i < m; i++) {
        for (int j = 0; j < n; j++) {
            char c = grid[i].charAt(j);
            if (c == '@') { startX = i; startY = j; }
            if (c >= 'a' && c <= 'f') totalKeys++;
        }
    }

    int[][][] dist = new int[m][n][1 << totalKeys];
    for (int i = 0; i < m; i++)
        for (int j = 0; j < n; j++)
            Arrays.fill(dist[i][j], Integer.MAX_VALUE);

    PriorityQueue<int[]> pq = new PriorityQueue<>((a,b) -> a[3] - b[3]);
    dist[startX][startY][0] = 0;
    pq.offer(new int[]{startX, startY, 0, 0}); // x,y,mask,steps

    int[][] dirs = {{0,1},{1,0},{0,-1},{-1,0}};

    while (!pq.isEmpty()) {
        int[] curr = pq.poll();
        int i = curr[0], j = curr[1], mask = curr[2], steps = curr[3];

        if (mask == (1 << totalKeys) - 1) return steps;

        for (int[] d : dirs) {
            int ni = i + d[0], nj = j + d[1];
            if (ni < 0 || ni >= m || nj < 0 || nj >= n) continue;

            char cell = grid[ni].charAt(nj);
            if (cell == '#') continue;

            int newMask = mask;
            if (cell >= 'a' && cell <= 'f') {
                newMask |= (1 << (cell - 'a'));
            }
            if (cell >= 'A' && cell <= 'F') {
                if ((newMask & (1 << (cell - 'A'))) == 0) continue;
            }

            if (steps + 1 < dist[ni][nj][newMask]) {
                dist[ni][nj][newMask] = steps + 1;
                pq.offer(new int[]{ni, nj, newMask, steps + 1});
```

```
                }
            }
        }
        return -1;
    }
```

■ 10 Practice Questions

m
st Path to Get All Keys](https://leetcode.com/problems/shortest-path-to-get-all-keys/)
st Path to Get All Keys (GFG)](https://www.geeksforgeeks.org/shortest-path-to-get-all-keys/)
um Moves to Collect All Keys](https://www.geeksforgeeks.org/minimum-moves-to-collect-all-keys-using-dijkstra/)
st Path with Keys and Doors](https://www.geeksforgeeks.org/shortest-path-with-keys-and-doors-using-dijkstra/)
um Steps to Reach End with Keys](https://www.geeksforgeeks.org/minimum-steps-to-reach-end-with-keys-using-dijkstra/)
st Path to Get All Keys in Matrix](https://www.geeksforgeeks.org/shortest-path-to-get-all-keys-in-matrix/)
um Time to Visit Disappearing Nodes](https://www.geeksforgeeks.org/minimum-time-to-visit-disappearing-nodes-using-dijkstra/)
st Path with Teleporters](https://www.geeksforgeeks.org/shortest-path-with-teleporters-using-dijkstra/)
um Cost to Reach Destination with Keys](https://www.geeksforgeeks.org/minimum-cost-to-reach-destination-with-keys-using-dijkstra/)
Coins with Minimum Steps](https://www.geeksforgeeks.org/collect-coins-with-minimum-steps-using-dijkstra/)

PATTERN 7: Dijkstra + Path Reconstruction

■ Theory

Not only find shortest distance, but also **reconstruct the actual path**. Store `parent[node]` for each node. After Dijkstra finishes, backtrack from destination to source.

Properties:

- Update parent whenever distance improves

- Path is reconstructed in reverse order

- Works only when there's a unique/single shortest path

■ Visual Diagram

```
Graph: 0--4--1--2--3
       |        |
       5--------
Parent array updates:
```

```
  dist[1] = 4, parent[1] = 0
  dist[2] = 6, parent[2] = 1
  dist[3] = 8, parent[3] = 2

Reconstruct path 0→3:
  start at 3, parent[3]=2
  parent[2]=1, parent[1]=0
  Reverse: 0→1→2→3
```

■ Template Code

```
public int[] dijkstraWithPath(int n, List<int[]>[] graph, int src, int dest) {
    int[] dist = new int[n];
    int[] parent = new int[n];
    Arrays.fill(dist, Integer.MAX_VALUE);
    Arrays.fill(parent, -1);
    dist[src] = 0;

    PriorityQueue<int[]> pq = new PriorityQueue<>((a,b) -> a[0] - b[0]);
    pq.offer(new int[]{0, src});

    while (!pq.isEmpty()) {
        int[] curr = pq.poll();
        int d = curr[0], u = curr[1];
        if (d > dist[u]) continue;

        for (int[] edge : graph[u]) {
            int v = edge[0], w = edge[1];
            if (dist[u] + w < dist[v]) {
                dist[v] = dist[u] + w;
                parent[v] = u;  // Store parent
                pq.offer(new int[]{dist[v], v});
            }
        }
    }

    // Reconstruct path
    List<Integer> path = new ArrayList<>();
    for (int v = dest; v != -1; v = parent[v]) {
        path.add(v);
    }
    Collections.reverse(path);
    return path.stream().mapToInt(i->i).toArray();
}
```

■ 10 Practice Questions

	Problem
	---
on	[Print Shortest Path](https://www.geeksforgeeks.org/problems/shortest-path-in-weighted-graph/1)
on	[Shortest Path Visiting All Nodes](https://leetcode.com/problems/shortest-path-visiting-all-nodes/)
on	[Minimum Weight Path in Graph](https://www.geeksforgeeks.org/minimum-weight-path-in-graph-using-dijkstra/)
on	[Shortest Path with Alternating Colors](https://leetcode.com/problems/shortest-path-with-alternating-colors/)
on	[Print All Shortest Paths](https://www.geeksforgeeks.org/print-all-shortest-paths-using-dijkstra/)

	Problem
on	[Find Shortest Path with Minimum Edges](https://www.geeksforgeeks.org/find-shortest-path-with-minimum-edges-using-dijkstra/)
on	[Shortest Path with Obstacles](https://www.geeksforgeeks.org/shortest-path-with-obstacles-using-dijkstra/)
on	[Minimum Cost Path with Path Printing](https://www.geeksforgeeks.org/minimum-cost-path-with-path-printing-using-dijkstra/)
on	[Find Path with Minimum Sum](https://www.geeksforgeeks.org/find-path-with-minimum-sum-using-dijkstra/)
on	[Shortest Path in Binary Matrix with Path](https://www.geeksforgeeks.org/shortest-path-in-binary-matrix-with-path-using-dijkstra/)

DIJKSTRA: Pattern Recognition Table

Pattern Name	Problem Signal	Key Insight	Time Complexity
---	---	---	---
Basic	"Shortest path weighted graph"	Standard PQ implementation	O((V+E) log V)
Multi-Source	"Nearest source", "Distance to any source"	Initialize PQ with all sources	O((V+E) log V)
Minimax	"Minimize maximum edge weight"	Store max edge so far, not sum	O((V+E) log V)
K Stops	"At most K stops", "Limited hops"	Add stop count to state	O(K × E log V)
Grid	"Grid with weights/costs"	Treat each cell as node	O(R×C log(R×C))
State	"Keys and doors", "Collect items"	State = (pos, bitmask)	O(V × 2^k × log(V × 2^k))
Path Reconstruction	"Print the path", "Find the route"	Store parent array	O((V+E) log V)

BELLMAN-FORD ALGORITHM - Complete Pattern Guide

■ **Properties & Complexity**

Property	Value
---	---
Time Complexity	O(V × E)
Space Complexity	O(V)
Works On	Weighted graphs with **NEGATIVE weights allowed**
Detects	Negative cycles
Finds	Shortest path from single source to all nodes

When to Use Bellman-Ford (The Signals)

Problem Signal	Pattern
----------	---------
"Negative weights present"	Pattern 1
"Detect negative cycle"	Pattern 2
"Cheapest flights with K stops"	Pattern 3
"Currency arbitrage detection"	Pattern 4
"Minimum cost with constraints"	Pattern 5
"Find if negative cycle exists"	Pattern 2

PATTERN 1: Basic Bellman-Ford (Single Source with Negative Weights)

■ Theory

Relax all edges **V-1 times**. After V-1 iterations, shortest paths are found (if no negative cycles). Why V-1? Longest simple path has V-1 edges.

Properties:

- Can handle negative weights

- Slower than Dijkstra but more general

- If graph has no negative cycles, V-1 iterations suffice

■ Visual Diagram

```
Graph: 0 → 1 (4), 0 → 2 (3), 2 → 1 (-2)

Iteration 1:
  dist[0] = 0
  Relax 0→1: dist[1] = min(∞, 0+4) = 4
  Relax 0→2: dist[2] = min(∞, 0+3) = 3
  Relax 2→1: dist[1] = min(4, 3+(-2)=1) = 1

Iteration 2:
  Relax 2→1: dist[1] = min(1, 3+(-2)=1) = 1

Result: dist[1] = 1 (found shorter path via 0→2→1)
```

■ Template Code

```
public int[] bellmanFord(int n, int[][] edges, int src) {
    int[] dist = new int[n];
    Arrays.fill(dist, Integer.MAX_VALUE);
    dist[src] = 0;

    // Relax V-1 times
    for (int i = 0; i < n - 1; i++) {
        for (int[] edge : edges) {
            int u = edge[0], v = edge[1], w = edge[2];
            if (dist[u] != Integer.MAX_VALUE && dist[u] + w < dist[v]) {
                dist[v] = dist[u] + w;
            }
        }
    }

    // Check for negative cycles
    for (int[] edge : edges) {
        int u = edge[0], v = edge[1], w = edge[2];
        if (dist[u] != Integer.MAX_VALUE && dist[u] + w < dist[v]) {
            return null;  // Negative cycle exists
        }
    }
    return dist;
}
```

■ 10 Practice Questions

ord Algorithm](https://www.geeksforgeeks.org/problems/distance-from-the-source-bellman-ford-algorithm/1)
elay Time](https://leetcode.com/problems/network-delay-time/)
ath with Negative Weights](https://www.geeksforgeeks.org/shortest-path-with-negative-weights-using-bellman-ford/)
ity With Smallest Neighbors](https://leetcode.com/problems/find-the-city-with-the-smallest-number-of-neighbors-at-a-threshold-distance
Cost to Reach Destination](https://www.geeksforgeeks.org/minimum-cost-to-reach-destination-using-bellman-ford/)
Flights Within K Stops](https://leetcode.com/problems/cheapest-flights-within-k-stops/)
Minimum Effort](https://leetcode.com/problems/path-with-minimum-effort/)
ath in Weighted Graph](https://www.geeksforgeeks.org/shortest-path-in-weighted-graph-using-bellman-ford/)
Time to Visit a Cell](https://www.geeksforgeeks.org/minimum-time-to-visit-a-cell-using-bellman-ford/)
ord Implementation](https://www.geeksforgeeks.org/bellman-ford-algorithm-dp-23/)

PATTERN 2: Negative Cycle Detection

■ Theory

After V-1 relaxations, perform **one more relaxation**. If any distance improves → negative cycle exists. A negative cycle means shortest path is undefined (-∞).

Properties:

- Negative cycle can be reached from source

- All nodes reachable from cycle have -∞ distance

- Used in arbitrage detection

■ Visual Diagram

```
Graph with negative cycle:
0 → 1 (1)
1 → 2 (1)
2 → 0 (-3)

Cycle: 0→1→2→0 sum = 1+1+(-3) = -1

Bellman-Ford after V-1=3 iterations:
  dist[0] = 0
  dist[1] = 1
  dist[2] = 2

4th iteration (check for negative cycle):
  Relax 2→0: dist[0] = min(0, 2+(-3)=-1) → IMPROVEMENT!
  Negative cycle detected!
```

■ Template Code

```
public boolean hasNegativeCycle(int n, int[][] edges, int src) {
    int[] dist = new int[n];
    Arrays.fill(dist, Integer.MAX_VALUE);
    dist[src] = 0;

    // Relax V times (instead of V-1)
    for (int i = 0; i < n; i++) {
        boolean updated = false;
        for (int[] edge : edges) {
            int u = edge[0], v = edge[1], w = edge[2];
            if (dist[u] != Integer.MAX_VALUE && dist[u] + w < dist[v]) {
                dist[v] = dist[u] + w;
                updated = true;
                if (i == n - 1) return true;  // Negative cycle on Vth iteration
            }
        }
        if (!updated) break;
    }
    return false;
}
```

■ 10 Practice Questions

	Problem	Pla
	---	---
on	[Negative Weight Cycle](https://www.geeksforgeeks.org/problems/negative-weight-cycle3504/1)	GF
on	[Detect Negative Cycle in Graph](https://www.geeksforgeeks.org/detect-negative-cycle-in-a-graph-using-bellman-ford/)	GF
on	[Find if Negative Cycle Exists](https://www.geeksforgeeks.org/find-if-negative-cycle-exists-using-bellman-ford/)	GF
on	[Minimum Cost with Negative Cycle](https://www.geeksforgeeks.org/minimum-cost-with-negative-cycle-using-bellman-ford/)	GF
on	[Arbitrage Detection](https://www.geeksforgeeks.org/detect-arbitrage-currency-exchange/)	GF
on	[Currency Arbitrage using Bellman-Ford](https://www.geeksforgeeks.org/currency-arbitrage-using-bellman-ford/)	GF
on	[Shortest Path with Negative Cycles](https://www.geeksforgeeks.org/shortest-path-with-negative-cycles-using-bellman-ford/)	GF
on	[Find Nodes in Negative Cycle](https://www.geeksforgeeks.org/find-nodes-in-negative-cycle-using-bellman-ford/)	GF
on	[Minimum Cycle Length in Graph](https://www.geeksforgeeks.org/minimum-cycle-length-in-graph-using-bellman-ford/)	GF
on	[Check if Graph has Negative Cycle](https://www.geeksforgeeks.org/check-if-graph-has-negative-cycle-using-bellman-ford/)	GF

PATTERN 3: K Stops / Limited Edges (Natural Bellman-Ford)

■ Theory

Bellman-Ford is **naturally suited** for "shortest path with at most K edges". Just run **K+1 iterations** instead of V-1. Each iteration adds at most one edge to paths.

Properties:

- No priority queue needed
- Natural stop count constraint
- Each iteration = paths with i edges

■ Visual Diagram

```
K=1 (max 2 edges):
Initial: dist = [0, ∞, ∞, ∞]

Iteration 1 (1 edge paths):
  dist[1] = 100, dist[2] = 500

Iteration 2 (2 edge paths):
  dist[3] = min(∞, dist[1]+500=600, dist[2]+100=600) = 600

Iteration 3 (3 edge paths - NOT allowed for K=1):
  dist[3] = min(600, dist[?]) → but we stop at K+1=2 iterations
```

■ Template Code

```
public int findCheapestPrice(int n, int[][] flights, int src, int dst, int k) {
    int[] dist = new int[n];
    Arrays.fill(dist, Integer.MAX_VALUE);
    dist[src] = 0;

    // Run K+1 iterations (K stops = K+1 edges)
    for (int i = 0; i <= k; i++) {
        int[] temp = Arrays.copyOf(dist, n);
        for (int[] flight : flights) {
            int u = flight[0], v = flight[1], w = flight[2];
            if (dist[u] != Integer.MAX_VALUE && dist[u] + w < temp[v]) {
                temp[v] = dist[u] + w;
            }
        }
        dist = temp;
    }

    return dist[dst] == Integer.MAX_VALUE ? -1 : dist[dst];
}
```

■ 10 Practice Questions

nts Within K Stops](https://leetcode.com/problems/cheapest-flights-within-k-stops/)
to Reach Destination with K Stops](https://www.geeksforgeeks.org/minimum-cost-to-reach-destination-with-k-stops-using-bellman-ford
with Exactly K Edges](https://www.geeksforgeeks.org/shortest-path-with-exactly-k-edges-using-bellman-ford/)
ys to Reach Destination with K Stops](https://www.geeksforgeeks.org/number-of-ways-to-reach-destination-with-k-stops-using-bellman
with K Stops](https://www.geeksforgeeks.org/minimum-time-with-k-stops-using-bellman-ford/)
te with K Stops](https://www.geeksforgeeks.org/cheapest-route-with-k-stops-using-bellman-ford/)
Cost with K Stops](https://www.geeksforgeeks.org/minimum-fuel-cost-with-k-stops-using-bellman-ford/)
with Maximum K Edges](https://www.geeksforgeeks.org/shortest-path-with-maximum-k-edges-using-bellman-ford/)
with K Stops (GFG)](https://www.geeksforgeeks.org/minimum-cost-with-k-stops-using-bellman-ford/)
Path with K Stops](https://www.geeksforgeeks.org/kth-smallest-path-with-k-stops-using-bellman-ford/)

PATTERN 4: Currency Arbitrage / Exchange Rates

■ Theory

Currency exchange rates can be modeled as a graph. Arbitrage exists if product of exchange rates in a cycle > 1. Take **negative log** to convert to negative cycle detection.

Properties:

- Convert rates to weights: `w = -log(rate)`

- Arbitrage = negative cycle

- Bellman-Ford detects it

■ Visual Diagram

```
Currencies: USD → EUR (0.9), EUR → GBP (0.8), GBP → USD (1.4)

Product: 0.9 × 0.8 × 1.4 = 1.008 > 1 → Arbitrage!

Convert to weights: w = -log(rate)
  USD→EUR: -log(0.9) = 0.105
  EUR→GBP: -log(0.8) = 0.223
  GBP→USD: -log(1.4) = -0.336

Sum of cycle = 0.105 + 0.223 + (-0.336) = -0.008 < 0 → Negative cycle!
```

■ Template Code

```
public boolean hasArbitrage(int n, double[][] exchangeRates) {
    // Convert to log weights
    double[][] graph = new double[n][n];
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            if (i != j && exchangeRates[i][j] > 0) {
                graph[i][j] = -Math.log(exchangeRates[i][j]);
            }
        }
    }

    // Bellman-Ford to detect negative cycle
    double[] dist = new double[n];
    Arrays.fill(dist, Double.POSITIVE_INFINITY);
    dist[0] = 0;

    for (int i = 0; i < n - 1; i++) {
        for (int u = 0; u < n; u++) {
            for (int v = 0; v < n; v++) {
                if (graph[u][v] > 0 && dist[u] + graph[u][v] < dist[v]) {
                    dist[v] = dist[u] + graph[u][v];
                }
            }
        }
    }

    // Check for negative cycle
    for (int u = 0; u < n; u++) {
        for (int v = 0; v < n; v++) {
            if (graph[u][v] > 0 && dist[u] + graph[u][v] < dist[v]) {
                return true;  // Arbitrage exists
            }
        }
    }
    return false;
}
```

■ 10 Practice Questions

Problem	Plat
---	---
[Currency Arbitrage](https://www.geeksforgeeks.org/detect-arbitrage-currency-exchange/)	GFG
[Find Arbitrage in Exchange Rates](https://www.geeksforgeeks.org/find-arbitrage-in-exchange-rates-using-bellman-ford/)	GFG
[Maximum Arbitrage Profit](https://www.geeksforgeeks.org/maximum-arbitrage-profit-using-bellman-ford/)	GFG
[Currency Exchange with Fees](https://www.geeksforgeeks.org/currency-exchange-with-fees-using-bellman-ford/)	GFG
[Detect Arbitrage in Forex](https://www.geeksforgeeks.org/detect-arbitrage-in-forex-using-bellman-ford/)	GFG
[Minimum Cost Currency Conversion](https://www.geeksforgeeks.org/minimum-cost-currency-conversion-using-bellman-ford/)	GFG
[Find Best Arbitrage Path](https://www.geeksforgeeks.org/find-best-arbitrage-path-using-bellman-ford/)	GFG
[Arbitrage in Cryptocurrency](https://www.geeksforgeeks.org/arbitrage-in-cryptocurrency-using-bellman-ford/)	GFG
[Multiple Currency Arbitrage](https://www.geeksforgeeks.org/multiple-currency-arbitrage-using-bellman-ford/)	GFG
[Maximum Return Currency Exchange](https://www.geeksforgeeks.org/maximum-return-currency-exchange-using-bellman-ford/)	GFG

PATTERN 5: Shortest Path with Constraints

■ Theory

When there are constraints like "path must have at most X red edges" or "must pass through certain nodes", use Bellman-Ford with **state** (node, constraint_value).

Properties:

- State space = nodes × (constraint_range)

- Bellman-Ford relaxes over state graph

- Similar to Dijkstra with state but handles negative weights

■ Template Code

```
public int shortestPathWithConstraints(int n, int[][] edges, int src, int dest, int maxRedEdges
    // dist[node][redUsed] = min distance
    int[][] dist = new int[n][maxRedEdges + 1];
    for (int[] row : dist) Arrays.fill(row, Integer.MAX_VALUE);
    dist[src][0] = 0;

    for (int i = 0; i < n - 1; i++) {
        for (int[] edge : edges) {
            int u = edge[0], v = edge[1], w = edge[2];
            boolean isRed = edge[3] == 1;
            for (int red = 0; red <= maxRedEdges; red++) {
```

```
                if (dist[u][red] == Integer.MAX_VALUE) continue;

                int newRed = red + (isRed ? 1 : 0);
                if (newRed <= maxRedEdges) {
                    if (dist[u][red] + w < dist[v][newRed]) {
                        dist[v][newRed] = dist[u][red] + w;
                    }
                }
            }
        }
    }

    int ans = Integer.MAX_VALUE;
    for (int red = 0; red <= maxRedEdges; red++) {
        ans = Math.min(ans, dist[dest][red]);
    }
    return ans == Integer.MAX_VALUE ? -1 : ans;
}
```

■ 10 Practice Questions

oblem	
------	-
hortest Path with At Most K Red Edges](https://www.geeksforgeeks.org/shortest-path-with-at-most-k-red-edges-using-bellman-ford/)	
inimum Cost with Color Constraints](https://www.geeksforgeeks.org/minimum-cost-with-color-constraints-using-bellman-ford/)	
hortest Path with Obstacle Constraints](https://www.geeksforgeeks.org/shortest-path-with-obstacle-constraints-using-bellman-ford/)	
ath with Maximum Probability](https://leetcode.com/problems/path-with-maximum-probability/)	
hortest Path with Time Constraint](https://www.geeksforgeeks.org/shortest-path-with-time-constraint-using-bellman-ford/)	
inimum Cost with Fuel Constraints](https://www.geeksforgeeks.org/minimum-cost-with-fuel-constraints-using-bellman-ford/)	
hortest Path with Toll Constraints](https://www.geeksforgeeks.org/shortest-path-with-toll-constraints-using-bellman-ford/)	
ath with Minimum Turns](https://www.geeksforgeeks.org/path-with-minimum-turns-using-bellman-ford/)	
hortest Path with Signal Constraints](https://www.geeksforgeeks.org/shortest-path-with-signal-constraints-using-bellman-ford/)	
inimum Time with Traffic Constraints](https://www.geeksforgeeks.org/minimum-time-with-traffic-constraints-using-bellman-ford/)	

BELLMAN-FORD: Pattern Recognition Table

Pattern Name	Problem Signal	Key Insight	Time Complexity
----------	----------	----------	----------
Basic	"Negative weights", "Shortest path"	V-1 relaxations	O(V × E)
Negative Cycle	"Detect arbitrage", "Is there negative cycle?"	Vth relaxation improves	O(V × E)
K Stops	"At most K edges", "Limited flights"	Run K+1 iterations	O(K × E)
Arbitrage	"Currency exchange", "Profit cycle"	Convert to -log weights	O(V × E)
Constraints	"Max red edges", "Toll limits"	Add constraint to state	O(V × C × E)

MINIMUM SPANNING TREE - Complete Pattern Guide

■ **Properties & Complexity**

Property	Kruskal	Prim
----------	---------	------
Time Complexity	O(E log E)	O((V+E) log V)
Space Complexity	O(V)	O(V)
Best For	Sparse graphs (E ≈ V)	Dense graphs (E ≈ V^2)
Approach	Edge-based (sort all)	Vertex-based (grow tree)
Data Structure	Union-Find	Priority Queue

When to Use MST (The Signals)

Problem Signal	Pattern
----------	---------
"Connect all points with minimum cost"	Pattern 1
"Minimum cost to connect all cities"	Pattern 1
"Find critical/pseudo-critical edges"	Pattern 2
"Second best MST"	Pattern 3
"Maximum spanning tree"	Pattern 4
"MST with constraints (max degree)"	Pattern 5
"Minimum cost to connect with given edges"	Pattern 6
"MST on complete graph with Manhattan distance"	Pattern 7

PATTERN 1: Basic MST (Kruskal + Prim)

■ Theory

Kruskal: Sort edges by weight, add smallest edges that don't create cycles (using Union-Find).

Prim: Start from one node, always add smallest edge connecting tree to new node (using PQ).

Properties:

- Tree with V-1 edges

- Minimum total weight

- Unique if all edge weights are distinct

■ Visual Diagram

```
Kruskal:
Edges sorted: (0-1:2), (2-3:3), (1-2:4), (0-3:5), (1-3:6)

Step 1: Add 0-1 (2) → {0,1}, {2}, {3}
Step 2: Add 2-3 (3) → {0,1}, {2,3}
Step 3: Add 1-2 (4) → {0,1,2,3} (all connected)
MST weight = 2+3+4=9
```

■ Template Code

```
// Kruskal's Algorithm
public int kruskalMST(int n, int[][] edges) { // edges: [u,v,w]
    Arrays.sort(edges, (a,b) -> a[2] - b[2]);
    DSU dsu = new DSU(n);
    int mstWeight = 0, edgesUsed = 0;

    for (int[] edge : edges) {
        if (dsu.union(edge[0], edge[1])) {
            mstWeight += edge[2];
            edgesUsed++;
            if (edgesUsed == n - 1) break;
        }
    }
    return edgesUsed == n - 1 ? mstWeight : -1;
}

// Prim's Algorithm
public int primMST(int n, List<int[]>[] graph) {
    boolean[] inMST = new boolean[n];
    PriorityQueue<int[]> pq = new PriorityQueue<>((a,b) -> a[1] - b[1]);
    pq.offer(new int[]{0, 0}); // {node, weight to connect}

    int mstWeight = 0, nodesInMST = 0;

    while (!pq.isEmpty() && nodesInMST < n) {
        int[] curr = pq.poll();
        int u = curr[0], w = curr[1];
        if (inMST[u]) continue;

        inMST[u] = true;
        mstWeight += w;
        nodesInMST++;
```

```
            for (int[] edge : graph[u]) {
                int v = edge[0], weight = edge[1];
                if (!inMST[v]) pq.offer(new int[]{v, weight});
            }
        }
        return nodesInMST == n ? mstWeight : -1;
    }
```

■ 10 Practice Questions

lem	P
--	--
mum Cost to Connect All Points](https://leetcode.com/problems/minimum-cost-to-connect-all-points/)	L
necting Cities With Minimum Cost](https://leetcode.com/problems/connecting-cities-with-minimum-cost/)	L
mum Spanning Tree](https://www.geeksforgeeks.org/problems/minimum-spanning-tree/1)	G
kal's Algorithm](https://www.geeksforgeeks.org/problems/kruskals-minimum-spanning-tree/1)	G
's Algorithm](https://www.geeksforgeeks.org/problems/prims-minimum-spanning-tree/1)	G
Cost to Connect All Nodes](https://leetcode.com/problems/min-cost-to-connect-all-nodes/)	L
ber of Operations to Make Network Connected](https://leetcode.com/problems/number-of-operations-to-make-network-connected/)	L
mum Cost to Connect Sticks](https://leetcode.com/problems/minimum-cost-to-connect-sticks/)	L
nect Ropes with Minimum Cost](https://www.geeksforgeeks.org/problems/minimum-cost-of-ropes/0)	G
mum Cost to Connect Cities](https://www.geeksforgeeks.org/minimum-cost-to-connect-cities-using-kruskal/)	G

PATTERN 2: Critical and Pseudo-Critical Edges in MST

■ Theory

Critical edge: Must be in every MST (if removed, MST weight increases)

Pseudo-critical edge: Can be in some MSTs but not all

Normal edge: Never in any MST

Properties:

- Critical: edge weight < all other edges forming cycle

- For each edge, check if it's in MST and if removing forces higher weight

■ Visual Diagram

```
Graph: 0--2--1, 0--3--1 (all edges weight 2)
```

```
MST weight = 4

Edge (0-1) weight 3:
  Without (0-1), MST = 4
  With (0-1) forced, need other edges: 0-1(3)+0-2(2)+0-3(2)=7 >4
  So (0-1) is NOT in any MST → normal edge
```

■ Template Code

```
public List<List<Integer>> findCriticalAndPseudoCriticalEdges(int n, int[][] edges) {
    int m = edges.length;
    // Add original index to edges
    int[][] edgesWithIdx = new int[m][4];
    for (int i = 0; i < m; i++) {
        System.arraycopy(edges[i], 0, edgesWithIdx[i], 0, 3);
        edgesWithIdx[i][3] = i;
    }

    // Sort by weight
    Arrays.sort(edgesWithIdx, (a,b) -> a[2] - b[2]);

    // Find normal MST weight
    int normalMSTWeight = kruskal(n, edgesWithIdx, -1, -1);

    List<Integer> critical = new ArrayList<>();
    List<Integer> pseudo = new ArrayList<>();

    for (int i = 0; i < m; i++) {
        // Check if critical: MST weight without this edge increases
        int withoutWeight = kruskal(n, edgesWithIdx, i, -1);
        if (withoutWeight > normalMSTWeight) {
            critical.add(i);
            continue;
        }

        // Check if pseudo: MST weight with this edge forced = normal weight
        int withWeight = kruskal(n, edgesWithIdx, -1, i);
        if (withWeight == normalMSTWeight) {
            pseudo.add(i);
        }
    }

    return Arrays.asList(critical, pseudo);
}

private int kruskal(int n, int[][] edges, int excludeIdx, int includeIdx) {
    DSU dsu = new DSU(n);
    int weight = 0, edgesUsed = 0;

    // Force include edge if specified
    if (includeIdx != -1) {
        int[] e = edges[includeIdx];
        dsu.union(e[0], e[1]);
        weight += e[2];
        edgesUsed++;
    }

    for (int[] e : edges) {
        if (e[3] == excludeIdx) continue;
        if (dsu.union(e[0], e[1])) {
            weight += e[2];
            edgesUsed++;
            if (edgesUsed == n - 1) break;
```

```
            }
        }
        return edgesUsed == n - 1 ? weight : Integer.MAX_VALUE;
    }
```

■ 10 Practice Questions

em	
-	
Critical and Pseudo-Critical Edges](https://leetcode.com/problems/find-critical-and-pseudo-critical-edges-in-minimum-spanning-tree/)	
al Connections in a Network](https://leetcode.com/problems/critical-connections-in-a-network/)	
num Spanning Tree with Critical Edges](https://www.geeksforgeeks.org/minimum-spanning-tree-with-critical-edges/)	
Bridges in Graph](https://www.geeksforgeeks.org/bridge-in-a-graph-using-tarjans-algorithm/)	
All Critical Edges](https://www.geeksforgeeks.org/find-all-critical-edges-in-a-graph/)	
num Edge Removal to Make MST](https://www.geeksforgeeks.org/minimum-edge-removal-to-make-mst-using-kruskal/)	
k if Edge is in All MSTs](https://www.geeksforgeeks.org/check-if-edge-is-in-all-msts-using-kruskal/)	
ber of Critical Edges in MST](https://www.geeksforgeeks.org/number-of-critical-edges-in-mst-using-kruskal/)	
Minimum Weight Edge in MST](https://www.geeksforgeeks.org/find-minimum-weight-edge-in-mst-using-kruskal/)	
Importance in MST](https://www.geeksforgeeks.org/edge-importance-in-mst-using-kruskal/)	

PATTERN 3: Second Best MST / K-th Best MST

■ Theory

Second best MST is the minimum weight spanning tree that is **different from the best MST**. Approach: For each edge in MST, remove it and find MST on remaining graph.

Properties:

- Second best MST weight > best MST weight (if unique)

- Can be found in O(E log V + V^2) using edge replacement

■ Visual Diagram

```
MST edges: e1, e2, e3 (weight sum=10)
Try removing e1 → find MST without e1 → weight=12
Try removing e2 → find MST without e2 → weight=11
Try removing e3 → find MST without e3 → weight=13
Second best = min(12,11,13) = 11
```

■ Template Code

```
public int secondBestMST(int n, int[][] edges) {
    // Find best MST first
    Arrays.sort(edges, (a,b) -> a[2] - b[2]);
    DSU dsu = new DSU(n);
    List<int[]> mstEdges = new ArrayList<>();
    int bestWeight = 0;

    for (int[] e : edges) {
        if (dsu.union(e[0], e[1])) {
            bestWeight += e[2];
            mstEdges.add(e);
        }
    }

    // Find second best by removing each MST edge
    int secondBest = Integer.MAX_VALUE;
    for (int[] skipEdge : mstEdges) {
        DSU dsu2 = new DSU(n);
        int weight = 0;
        int edgesUsed = 0;

        for (int[] e : edges) {
            if (e == skipEdge) continue;
            if (dsu2.union(e[0], e[1])) {
                weight += e[2];
                edgesUsed++;
                if (edgesUsed == n - 1) break;
            }
        }
        if (edgesUsed == n - 1) {
            secondBest = Math.min(secondBest, weight);
        }
    }
    return secondBest;
}
```

■ 10 Practice Questions

m
d Best MST](https://www.geeksforgeeks.org/problems/second-minimum-spanning-tree/1)
mallest MST](https://www.geeksforgeeks.org/k-th-smallest-mst-using-kruskal/)
um Spanning Tree with Alternate Edges](https://www.geeksforgeeks.org/minimum-spanning-tree-with-alternate-edges/)
econd Minimum Spanning Tree](https://www.geeksforgeeks.org/find-second-minimum-spanning-tree-using-kruskal/)
er of Minimum Spanning Trees](https://www.geeksforgeeks.org/number-of-minimum-spanning-trees-using-kruskal/)
est MST using Prim](https://www.geeksforgeeks.org/k-th-best-mst-using-prim/)
d Best MST with Duplicate Weights](https://www.geeksforgeeks.org/second-best-mst-with-duplicate-weights/)
um Spanning Tree with Constraints](https://www.geeksforgeeks.org/minimum-spanning-tree-with-constraints/)
ll Minimum Spanning Trees](https://www.geeksforgeeks.org/find-all-minimum-spanning-trees-using-kruskal/)
um Weight Spanning Tree with Exactly K Edges](https://www.geeksforgeeks.org/minimum-weight-spanning-tree-with-exactly-k-edges/)

PATTERN 4: Maximum Spanning Tree

■ Theory

Same as MST but with **maximum total weight**. Simply sort edges in **descending** order (Kruskal) or use negative weights (Prim).

Properties:

- Use negative weights in Prim

- Or sort edges descending in Kruskal

- Applications: Maximum bandwidth, maximum reliability

■ Visual Diagram

```
Edges sorted descending: (1-2:9), (0-1:8), (0-2:7), (2-3:6), (1-3:5)
Step 1: Add 1-2 (9)
Step 2: Add 0-1 (8)
Step 3: Add 2-3 (6) → all connected
Max MST weight = 9+8+6=23
```

■ Template Code

```
public int maximumSpanningTree(int n, int[][] edges) {
    // Sort in descending order
    Arrays.sort(edges, (a,b) -> b[2] - a[2]);
    DSU dsu = new DSU(n);
    int maxWeight = 0, edgesUsed = 0;

    for (int[] edge : edges) {
        if (dsu.union(edge[0], edge[1])) {
            maxWeight += edge[2];
            edgesUsed++;
            if (edgesUsed == n - 1) break;
        }
    }
    return edgesUsed == n - 1 ? maxWeight : -1;
}
```

■ 10 Practice Questions

blem	
----	-
ximum Spanning Tree](https://www.geeksforgeeks.org/problems/maximum-spanning-tree/1)	

blem	
ximum Cost to Connect All Points](https://www.geeksforgeeks.org/maximum-cost-to-connect-all-points/)	
ximum Bandwidth Path](https://www.geeksforgeeks.org/maximum-bandwidth-path-using-maximum-spanning-tree/)	
ximum Reliability Network](https://www.geeksforgeeks.org/maximum-reliability-network-using-maximum-spanning-tree/)	
ximum Weight Spanning Tree using Prim](https://www.geeksforgeeks.org/maximum-weight-spanning-tree-using-prim/)	
nnect Cities with Maximum Cost](https://www.geeksforgeeks.org/connect-cities-with-maximum-cost-using-maximum-spanning-tree/)	
ximum Product Spanning Tree](https://www.geeksforgeeks.org/maximum-product-spanning-tree-using-maximum-spanning-tree/)	
ximum Spanning Tree in Complete Graph](https://www.geeksforgeeks.org/maximum-spanning-tree-in-complete-graph/)	
d Maximum Spanning Tree using Kruskal](https://www.geeksforgeeks.org/find-maximum-spanning-tree-using-kruskal/)	
ximum Cost to Connect Sticks](https://www.geeksforgeeks.org/maximum-cost-to-connect-sticks-using-maximum-spanning-tree/)	

PATTERN 5: MST with Degree Constraints

■ Theory

Find MST where a specific node has **at most K edges** (degree constraint). Remove the node, find MST on remaining graph, then connect the node with K smallest edges.

Properties:

- NP-hard for general degree constraints

- For single node, solvable in O(E log E)

■ Visual Diagram

```
Node 0 has degree constraint ≤ 2
Without 0: MST on {1,2,3}
Then connect 0 to its 2 smallest edges to this MST
```

■ Template Code

```
public int mstWithDegreeConstraint(int n, int[][] edges, int constrainedNode, int maxDegree) {
    // Separate edges connected to constrained node
    List<int[]> nodeEdges = new ArrayList<>();
    List<int[]> otherEdges = new ArrayList<>();

    for (int[] e : edges) {
        if (e[0] == constrainedNode || e[1] == constrainedNode) {
            nodeEdges.add(e);
        } else {
            otherEdges.add(e);
        }
    }
```

```
        // Sort edges not connected to constrained node
        otherEdges.sort((a,b) -> a[2] - b[2]);
        DSU dsu = new DSU(n);
        int weight = 0, edgesUsed = 0;

        // Build MST on graph without constrained node
        for (int[] e : otherEdges) {
            if (dsu.union(e[0], e[1])) {
                weight += e[2];
                edgesUsed++;
            }
        }

        // Connect constrained node with smallest edges
        nodeEdges.sort((a,b) -> a[2] - b[2]);
        int degree = 0;
        for (int[] e : nodeEdges) {
            if (degree < maxDegree && dsu.union(e[0], e[1])) {
                weight += e[2];
                degree++;
                edgesUsed++;
            }
        }

        return edgesUsed == n - 1 ? weight : -1;
    }
```

■ 10 Practice Questions

h Degree Constraint](https://www.geeksforgeeks.org/minimum-spanning-tree-with-degree-constraint/)
Constrained MST](https://www.geeksforgeeks.org/degree-constrained-minimum-spanning-tree/)
m Spanning Tree with Maximum Degree](https://www.geeksforgeeks.org/minimum-spanning-tree-with-maximum-degree-using-kruskal/)
d Degree MST](https://www.geeksforgeeks.org/bounded-degree-minimum-spanning-tree-using-prim/)
h Node Degree Limit](https://www.geeksforgeeks.org/mst-with-node-degree-limit-using-kruskal/)
m Cost with Degree Constraints](https://www.geeksforgeeks.org/minimum-cost-with-degree-constraints-using-mst/)
Connected MST](https://www.geeksforgeeks.org/k-edge-connected-minimum-spanning-tree/)
h Maximum Node Degree](https://www.geeksforgeeks.org/mst-with-maximum-node-degree-using-prim/)
Constrained MST using Kruskal](https://www.geeksforgeeks.org/degree-constrained-mst-using-kruskal/)
m Spanning Tree with Degree Bound](https://www.geeksforgeeks.org/minimum-spanning-tree-with-degree-bound-using-prim/)

PATTERN 6: MST with Pre-existing Edges / Forced Edges

■ Theory

Some edges are **already built** (cost 0) or **forced** to be in MST. Start DSU with these edges pre-unioned, then run standard MST.

Properties:

- Union forced edges first

- If forced edges create cycle → impossible

- Then add remaining edges

■ Visual Diagram

```
Forced edges: (0-1), (2-3)
Start DSU: {0,1}, {2,3}
Then add smallest edges connecting these components
```

■ Template Code

```
public int mstWithForcedEdges(int n, int[][] allEdges, int[][] forcedEdges) {
    DSU dsu = new DSU(n);
    int weight = 0;
    int edgesUsed = 0;

    // Add forced edges first
    for (int[] e : forcedEdges) {
        if (!dsu.union(e[0], e[1])) {
            return -1;  // Forced edge creates cycle
        }
        weight += e[2];
        edgesUsed++;
    }

    // Sort remaining edges
    List<int[]> remaining = new ArrayList<>();
    for (int[] e : allEdges) remaining.add(e);
    remaining.sort((a,b) -> a[2] - b[2]);

    // Regular Kruskal
    for (int[] e : remaining) {
        if (dsu.union(e[0], e[1])) {
            weight += e[2];
            edgesUsed++;
            if (edgesUsed == n - 1) break;
        }
    }

    return edgesUsed == n - 1 ? weight : -1;
}
```

■ 10 Practice Questions

re-existing Edges](https://www.geeksforgeeks.org/minimum-spanning-tree-with-pre-existing-edges/)
ost to Connect with Some Existing Edges](https://www.geeksforgeeks.org/minimum-cost-to-connect-with-some-existing-edges-using-m
ost to Connect with Prebuilt Roads](https://www.geeksforgeeks.org/minimum-cost-to-connect-with-prebuilt-roads-using-mst/)
Mandatory Edges](https://www.geeksforgeeks.org/mst-with-mandatory-edges-using-kruskal/)
ties with Some Roads Already Built](https://www.geeksforgeeks.org/connect-cities-with-some-roads-already-built-using-mst/)
panning Tree with Fixed Edges](https://www.geeksforgeeks.org/minimum-spanning-tree-with-fixed-edges-using-kruskal/)
Required Edges](https://www.geeksforgeeks.org/mst-with-required-edges-using-prim/)
ost to Connect with Existing Connections](https://www.geeksforgeeks.org/minimum-cost-to-connect-with-existing-connections-using-ms
ome Edges Already Chosen](https://www.geeksforgeeks.org/mst-with-some-edges-already-chosen-using-kruskal/)
ost to Build with Pre-built Paths](https://www.geeksforgeeks.org/minimum-cost-to-build-with-pre-built-paths-using-mst/)

PATTERN 7: MST on Manhattan Distance / Complete Graph

■ Theory

When connecting points with Manhattan distance, you don't need all O(n²) edges. Only consider **neighbors in sorted order** (by x and y coordinates).

Properties:

- For Manhattan distance, only O(n) edges needed

- Sort points by x, connect consecutive points

- Sort points by y, connect consecutive points

- Then run Kruskal

■ Visual Diagram

```
Points: (0,0), (1,2), (3,1), (2,3)

Sort by x: (0,0)-(1,2) dist=3, (1,2)-(2,3) dist=2, (2,3)-(3,1) dist=3
Sort by y: (0,0)-(3,1) dist=4, (3,1)-(1,2) dist=3, (1,2)-(2,3) dist=2
Total unique edges to consider: O(n) instead of O(n²)
```

■ Template Code

```
public int minCostConnectPoints(int[][] points) {
    int n = points.length;
    List<int[]> edges = new ArrayList<>();
    // Connect consecutive points by x-coordinate
```

```
        Integer[] idx = new Integer[n];
        for (int i = 0; i < n; i++) idx[i] = i;
        Arrays.sort(idx, (a,b) -> points[a][0] - points[b][0]);
        for (int i = 0; i < n - 1; i++) {
            int dist = Math.abs(points[idx[i]][0] - points[idx[i+1]][0]) +
                       Math.abs(points[idx[i]][1] - points[idx[i+1]][1]);
            edges.add(new int[]{idx[i], idx[i+1], dist});
        }

        // Connect consecutive points by y-coordinate
        Arrays.sort(idx, (a,b) -> points[a][1] - points[b][1]);
        for (int i = 0; i < n - 1; i++) {
            int dist = Math.abs(points[idx[i]][0] - points[idx[i+1]][0]) +
                       Math.abs(points[idx[i]][1] - points[idx[i+1]][1]);
            edges.add(new int[]{idx[i], idx[i+1], dist});
        }

        // Kruskal on these O(n) edges
        edges.sort((a,b) -> a[2] - b[2]);
        DSU dsu = new DSU(n);
        int total = 0, edgesUsed = 0;

        for (int[] e : edges) {
            if (dsu.union(e[0], e[1])) {
                total += e[2];
                edgesUsed++;
                if (edgesUsed == n - 1) break;
            }
        }
        return total;
    }
```

■ 10 Practice Questions

to Connect All Points](https://leetcode.com/problems/minimum-cost-to-connect-all-points/)
to Connect All Points with Manhattan Distance](https://www.geeksforgeeks.org/minimum-cost-to-connect-all-points-with-manhattan-dist
s with Minimum Manhattan Distance](https://www.geeksforgeeks.org/connect-points-with-minimum-manhattan-distance-using-mst/)
ning Tree with Manhattan Distance](https://www.geeksforgeeks.org/minimum-spanning-tree-with-manhattan-distance-using-kruskal/)
to Connect Points in Grid](https://www.geeksforgeeks.org/cheapest-way-to-connect-points-in-grid-using-mst/)
to Connect Cities with Manhattan Distance](https://www.geeksforgeeks.org/minimum-cost-to-connect-cities-with-manhattan-distance-us
ints with Minimum Cost using Manhattan](https://www.geeksforgeeks.org/connect-all-points-with-minimum-cost-using-manhattan-using-
nce to Connect Points](https://www.geeksforgeeks.org/minimum-distance-to-connect-points-using-mst/)
to Connect Points in 2D Plane](https://www.geeksforgeeks.org/minimum-cost-to-connect-points-in-2d-plane-using-mst/)
s with Minimum Euclidean Distance](https://www.geeksforgeeks.org/connect-points-with-minimum-euclidean-distance-using-mst/)

MST: Pattern Recognition Table

Pattern Name	Problem Signal	Key Insight	Algorithm
----------	----------	----------	----------
Basic MST	"Connect all with min cost"	Standard Kruskal/Prim	O(E log E)
Critical Edges	"Find edges that must be in all MSTs"	Check MST weight without edge	O(E² log E)
Second Best	"Find second minimum spanning tree"	Remove each MST edge and recompute	O(E × V log V)
Maximum MST	"Maximum cost to connect"	Sort descending	O(E log E)
Degree Constraint	"Node can have at most K connections"	Remove node, then reconnect	O(E log E)
Forced Edges	"Some edges already built"	Union forced edges first	O(E log E)
Manhattan	"Points in 2D grid"	Only connect nearest neighbors	O(n log n)

TOPOLOGICAL SORT - Complete Pattern Guide

■ **Properties & Complexity**

Property	Value
----------	-------
Time Complexity	O(V + E)
Space Complexity	O(V)
Works On	Directed Acyclic Graph (DAG)
Applications	Task scheduling, course prerequisites, build systems

When to Use Topological Sort (The Signals)

Problem Signal	Pattern
----------	---------
"Course schedule / prerequisites"	Pattern 1
"Build order / compilation order"	Pattern 1
"Alien dictionary character order"	Pattern 2
"Longest path in DAG"	Pattern 3
"All topological orders"	Pattern 4
"Minimum time to complete tasks"	Pattern 5
"Task scheduling with cooldown"	Pattern 6
"Parallel task execution"	Pattern 7

Problem Signal	Pattern
"Find if ordering is unique"	Pattern 8
"Reconstruct sequence from pairs"	Pattern 9

PATTERN 1: Basic Topological Sort (Kahn's Algorithm / DFS)

■ Theory

Order nodes such that for every edge u→v, u appears before v. Two approaches:

Kahn's (BFS): Remove nodes with in-degree 0 repeatedly.

DFS: Process nodes in post-order, reverse the result.

Properties:

- Only possible on DAG

- If cycle exists, not all nodes will be processed

■ Visual Diagram

```
DAG: 0→1→3, 0→2→3, 2→4

Kahn's:
In-degrees: 0:0, 1:1, 2:1, 3:2, 4:1
Queue: [0]
Remove 0 → decrease in-degrees: 1:0, 2:0 → queue [1,2]
Remove 1 → decrease in-degree of 3: 3:1
Remove 2 → decrease in-degree of 3: 3:0, 4:0 → queue [3,4]
Remove 3, then 4
Order: 0,1,2,3,4
```

■ Template Code

```
// Kahn's Algorithm (BFS)
public int[] topologicalSortKahn(int n, List<List<Integer>> graph) {
    int[] inDegree = new int[n];
    for (int u = 0; u < n; u++) {
        for (int v : graph.get(u)) {
            inDegree[v]++;
        }
    }

    Queue<Integer> queue = new LinkedList<>();
    for (int i = 0; i < n; i++) {
        if (inDegree[i] == 0) queue.offer(i);
    }
    int[] result = new int[n];
```

```
        int idx = 0;
        while (!queue.isEmpty()) {
            int u = queue.poll();
            result[idx++] = u;
            for (int v : graph.get(u)) {
                inDegree[v]--;
                if (inDegree[v] == 0) queue.offer(v);
            }
        }

        return idx == n ? result : new int[0];  // Empty if cycle exists
    }

    // DFS-based topological sort
    public int[] topologicalSortDFS(int n, List<List<Integer>> graph) {
        boolean[] visited = new boolean[n];
        boolean[] inStack = new boolean[n];
        Stack<Integer> stack = new Stack<>();

        for (int i = 0; i < n; i++) {
            if (!visited[i]) {
                if (!dfs(i, graph, visited, inStack, stack)) {
                    return new int[0];  // Cycle detected
                }
            }
        }

        int[] result = new int[n];
        for (int i = 0; i < n; i++) {
            result[i] = stack.pop();
        }
        return result;
    }

    private boolean dfs(int u, List<List<Integer>> graph, boolean[] visited,
                        boolean[] inStack, Stack<Integer> stack) {
        visited[u] = true;
        inStack[u] = true;

        for (int v : graph.get(u)) {
            if (!visited[v]) {
                if (!dfs(v, graph, visited, inStack, stack)) return false;
            } else if (inStack[v]) {
                return false;  // Cycle detected
            }
        }

        inStack[u] = false;
        stack.push(u);
        return true;
    }
```

■ 10 Practice Questions

oblem	Plat
------	------
ourse Schedule II](https://leetcode.com/problems/course-schedule-ii/)	Lee
opological Sort](https://www.geeksforgeeks.org/problems/topological-sort/1)	GFG
nd the Order of Characters from Dictionary](https://www.geeksforgeeks.org/problems/alien-dictionary/1)	GFG

oblem	Plat
ourse Schedule](https://leetcode.com/problems/course-schedule/)	Lee
ahn's Algorithm](https://www.geeksforgeeks.org/problems/kahns-algorithm/1)	GFG
arallel Courses](https://leetcode.com/problems/parallel-courses/)	Lee
nd the Order of Tasks](https://www.geeksforgeeks.org/problems/find-the-order-of-tasks/1)	GFG
uild Dependencies](https://www.geeksforgeeks.org/problems/build-dependencies/1)	GFG
inimum Time to Complete All Tasks](https://www.geeksforgeeks.org/minimum-time-to-complete-all-tasks-using-topological-sort/)	GFG
equence Reconstruction](https://leetcode.com/problems/sequence-reconstruction/)	Lee

---

PATTERN 2: Alien Dictionary (Character Ordering)

■ Theory

Given sorted words in an alien language, find the order of characters. Build graph where each adjacent word pair gives edge from first differing character to second.

Properties:

- Graph nodes = characters (26 letters)

- Edge u→v means u comes before v in alien alphabet

- Use topological sort on this graph

■ Visual Diagram

```
Words: ["wrt", "wrf", "er", "ett", "rftt"]
Compare "wrt" vs "wrf": 't' → 'f' (t before f)
Compare "wrf" vs "er": 'w' → 'e' (w before e)
Compare "er" vs "ett": 'r' → 't' (r before t)
Compare "ett" vs "rftt": 'e' → 'r' (e before r)

Graph: t→f, w→e, r→t, e→r
Topological order: w, e, r, t, f
```

■ Template Code

```
public String alienOrder(String[] words) {
    // Build graph
    Map<Character, Set<Character>> graph = new HashMap<>();
    Map<Character, Integer> inDegree = new HashMap<>();

    // Initialize
    for (String word : words) {
        for (char c : word.toCharArray()) {
```

```
                graph.putIfAbsent(c, new HashSet<>());
                inDegree.putIfAbsent(c, 0);
            }
        }

        // Build edges
        for (int i = 0; i < words.length - 1; i++) {
            String w1 = words[i], w2 = words[i+1];
            int minLen = Math.min(w1.length(), w2.length());
            boolean found = false;

            for (int j = 0; j < minLen; j++) {
                char c1 = w1.charAt(j), c2 = w2.charAt(j);
                if (c1 != c2) {
                    if (!graph.get(c1).contains(c2)) {
                        graph.get(c1).add(c2);
                        inDegree.put(c2, inDegree.get(c2) + 1);
                    }
                    found = true;
                    break;
                }
            }

            // Check invalid case: "abc", "ab"
            if (!found && w1.length() > w2.length()) return "";
        }

        // Topological sort
        Queue<Character> queue = new LinkedList<>();
        for (char c : inDegree.keySet()) {
            if (inDegree.get(c) == 0) queue.offer(c);
        }

        StringBuilder sb = new StringBuilder();
        while (!queue.isEmpty()) {
            char c = queue.poll();
            sb.append(c);
            for (char next : graph.get(c)) {
                inDegree.put(next, inDegree.get(next) - 1);
                if (inDegree.get(next) == 0) queue.offer(next);
            }
        }

        return sb.length() == inDegree.size() ? sb.toString() : "";
    }
```

■ 10 Practice Questions

blem	Platform
---	----------
en Dictionary](https://leetcode.com/problems/alien-dictionary/)	LeetCode
en Dictionary (GFG)](https://www.geeksforgeeks.org/problems/alien-dictionary/1)	GFG
d Order of Characters](https://www.geeksforgeeks.org/find-order-of-characters-using-topological-sort/)	GFG
rifying Alien Dictionary](https://leetcode.com/problems/verifying-an-alien-dictionary/)	LeetCode
en Language Order](https://www.geeksforgeeks.org/alien-language-order-using-topological-sort/)	GFG
aracter Order in Alien Language](https://www.geeksforgeeks.org/character-order-in-alien-language-using-topological-sort/)	GFG

blem	Platform
d Alien Dictionary Order](https://www.geeksforgeeks.org/find-alien-dictionary-order-using-topological-sort/)	GFG
en Dictionary with Invalid Cases](https://www.geeksforgeeks.org/alien-dictionary-with-invalid-cases/)	GFG
ltiple Alien Dictionaries](https://www.geeksforgeeks.org/multiple-alien-dictionaries-using-topological-sort/)	GFG
en Dictionary from Sorted Words](https://www.geeksforgeeks.org/alien-dictionary-from-sorted-words-using-topological-sort/)	GFG

PATTERN 3: Longest Path in DAG

■ Theory

In a DAG, longest path can be found using **DP on topological order**. Process nodes in topological order, update neighbors: `dp[v] = max(dp[v], dp[u] + weight(u→v))`.

Properties:

- Works only on DAG (no cycles)

- O(V + E) time complexity

- Also gives shortest path in DAG

■ Visual Diagram

```
DAG: 0→1(5), 0→2(3), 1→3(2), 2→3(4)

Topological order: 0,1,2,3
dp[0]=0
Process 0: dp[1]=5, dp[2]=3
Process 1: dp[3]=max(0,5+2=7)=7
Process 2: dp[3]=max(7,3+4=7)=7
Longest path: 0→1→3 weight=7
```

■ Template Code

```
public int longestPathInDAG(int n, List<int[]>[] graph) {
    // Get topological order
    int[] topo = topologicalSortKahn(n, graph);

    int[] dp = new int[n];
    int longest = 0;

    for (int u : topo) {
        for (int[] edge : graph[u]) {
            int v = edge[0], w = edge[1];
            if (dp[u] + w > dp[v]) {
                dp[v] = dp[u] + w;
                longest = Math.max(longest, dp[v]);
```

```
                }
            }
        }
        return longest;
    }
```

■ 10 Practice Questions

Problem	Pla
---	---
[Longest Path in DAG](https://www.geeksforgeeks.org/problems/longest-path-in-a-dag/1)	GF
[Longest Increasing Path in Matrix](https://leetcode.com/problems/longest-increasing-path-in-a-matrix/)	Le
[Minimum Time to Complete Tasks](https://www.geeksforgeeks.org/minimum-time-to-complete-tasks-using-topological-sort/)	GF
[Maximum Path Sum in DAG](https://www.geeksforgeeks.org/maximum-path-sum-in-dag-using-topological-sort/)	GF
[Longest Path in Directed Graph](https://www.geeksforgeeks.org/longest-path-in-directed-graph-using-topological-sort/)	GF
[Critical Path Method](https://www.geeksforgeeks.org/critical-path-method-using-topological-sort/)	GF
[Longest Distance in DAG](https://www.geeksforgeeks.org/longest-distance-in-dag-using-topological-sort/)	GF
[Maximum Length Chain](https://leetcode.com/problems/maximum-length-of-pair-chain/)	Le
[Longest Path with Different Weights](https://www.geeksforgeeks.org/longest-path-with-different-weights-using-topological-sort/)	GF
[Longest Path in a Grid DAG](https://www.geeksforgeeks.org/longest-path-in-a-grid-dag-using-topological-sort/)	GF

PATTERN 4: All Topological Orders (Backtracking)

■ Theory

When there are **multiple valid topological orders**, we need to generate all of them. Use backtracking with in-degree tracking.

Properties:

- Exponential number of possibilities (O(n!))

- Used in constraint satisfaction problems

■ Visual Diagram

```
Graph: 0→1, 0→2
Possible topological orders:
  [0,1,2]
  [0,2,1]
```

■ Template Code

```
public List<List<Integer>> allTopologicalOrders(int n, List<List<Integer>> graph) {
    int[] inDegree = new int[n];
    for (int u = 0; u < n; u++) {
        for (int v : graph.get(u)) inDegree[v]++;
    }

    List<List<Integer>> result = new ArrayList<>();
    List<Integer> current = new ArrayList<>();
    boolean[] visited = new boolean[n];

    backtrack(n, graph, inDegree, visited, current, result);
    return result;
}

private void backtrack(int n, List<List<Integer>> graph, int[] inDegree,
                       boolean[] visited, List<Integer> current, List<List<Integer>> result) {
    if (current.size() == n) {
        result.add(new ArrayList<>(current));
        return;
    }

    for (int u = 0; u < n; u++) {
        if (!visited[u] && inDegree[u] == 0) {
            // Choose u
            visited[u] = true;
            current.add(u);

            // Reduce in-degree of neighbors
            for (int v : graph.get(u)) inDegree[v]--;

            backtrack(n, graph, inDegree, visited, current, result);

            // Undo
            visited[u] = false;
            current.remove(current.size() - 1);
            for (int v : graph.get(u)) inDegree[v]++;
        }
    }
}
```

■ 10 Practice Questions

ological Sorts](https://www.geeksforgeeks.org/problems/all-topological-sorts/1)
Topological Orders](https://www.geeksforgeeks.org/print-all-topological-orders-using-backtracking/)
Possible Orders](https://www.geeksforgeeks.org/find-all-possible-orders-using-topological-sort/)
of Topological Orders](https://www.geeksforgeeks.org/number-of-topological-orders-using-dp/)
opological Sorts](https://www.geeksforgeeks.org/count-topological-sorts-using-dp/)
Task Orders](https://www.geeksforgeeks.org/all-valid-task-orders-using-topological-sort/)
te All Course Schedules](https://www.geeksforgeeks.org/generate-all-course-schedules-using-topological-sort/)
sible Build Orders](https://www.geeksforgeeks.org/all-possible-build-orders-using-topological-sort/)

raphically Smallest Topological Order](https://www.geeksforgeeks.org/lexicographically-smallest-topological-order-using-topological-sor
ological Order](https://www.geeksforgeeks.org/k-th-topological-order-using-topological-sort/)

PATTERN 5: Minimum Time to Complete Tasks (Parallel Execution)

■ Theory

When tasks can be executed in **parallel** (unlimited workers), the minimum time is the longest path from start to node in terms of task duration. Each node's completion time = max(parent completion times) + node's duration.

Properties:

- Process in topological order

- Each node's time = max(incoming) + duration

- Total time = max of all node times

■ Visual Diagram

```
Tasks with duration: 0(2), 1(3), 2(1), 3(4)
Edges: 0→1, 0→2, 1→3, 2→3

Topological order: 0,1,2,3
time[0] = 2
time[1] = 2+3=5
time[2] = 2+1=3
time[3] = max(5,3)+4=9
Total time = 9
```

■ Template Code

```
public int minimumTimeToCompleteTasks(int n, List<int[]>[] graph, int[] duration) {
    int[] inDegree = new int[n];
    for (int u = 0; u < n; u++) {
        for (int[] edge : graph[u]) inDegree[edge[0]]++;
    }

    Queue<Integer> queue = new LinkedList<>();
    int[] finishTime = new int[n];

    for (int i = 0; i < n; i++) {
        if (inDegree[i] == 0) {
            queue.offer(i);
```

```
                finishTime[i] = duration[i];
            }
        }

        while (!queue.isEmpty()) {
            int u = queue.poll();
            for (int[] edge : graph[u]) {
                int v = edge[0];
                finishTime[v] = Math.max(finishTime[v], finishTime[u] + duration[v]);
                inDegree[v]--;
                if (inDegree[v] == 0) queue.offer(v);
            }
        }

        int maxTime = 0;
        for (int t : finishTime) maxTime = Math.max(maxTime, t);
        return maxTime;
    }
```

■ 10 Practice Questions

Complete All Tasks](https://www.geeksforgeeks.org/minimum-time-to-complete-all-tasks-using-topological-sort/)
https://leetcode.com/problems/parallel-courses/)
Complete Tasks with Dependencies](https://www.geeksforgeeks.org/minimum-time-to-complete-tasks-with-dependencies-using-topolog
of Semesters](https://leetcode.com/problems/parallel-courses-ii/)
with Parallel Processing](https://www.geeksforgeeks.org/task-scheduling-with-parallel-processing-using-topological-sort/)
Finish All Jobs](https://www.geeksforgeeks.org/minimum-time-to-finish-all-jobs-using-topological-sort/)
Execute All Tasks](https://www.geeksforgeeks.org/minimum-time-to-execute-all-tasks-using-topological-sort/)
cution with Dependencies](https://www.geeksforgeeks.org/parallel-task-execution-with-dependencies-using-topological-sort/)
Compile All Modules](https://www.geeksforgeeks.org/minimum-time-to-compile-all-modules-using-topological-sort/)
Build with Dependencies](https://www.geeksforgeeks.org/minimum-time-to-build-with-dependencies-using-topological-sort/)

PATTERN 6: Task Scheduling with Cooldown / Resource Constraints

■ Theory

Tasks have cooldown periods or resource constraints after execution. Model as **state-space search** with time dimension.

Properties:

- Add cooldown state to each node

- Use BFS/DP on state (node, cooldown_remaining)

■ Template Code

```
public int scheduleWithCooldown(int n, List<Integer>[] graph, int[] cooldown) {
    // dist[node][cd] = earliest time to finish node with cd remaining
    int[][] dist = new int[n][100]; // 100 = max cooldown
    for (int[] row : dist) Arrays.fill(row, Integer.MAX_VALUE);

    PriorityQueue<int[]> pq = new PriorityQueue<>((a,b) -> a[0] - b[0]);
    // {time, node, cooldown_remaining}

    // Implementation depends on specific constraints
    return 0;
}
```

■ 10 Practice Questions

uler](https://leetcode.com/problems/task-scheduler/)
uler with Cooldown](https://www.geeksforgeeks.org/task-scheduler-with-cooldown-using-topological-sort/)
me to Complete Tasks with Cooldown](https://www.geeksforgeeks.org/minimum-time-to-complete-tasks-with-cooldown-using-topologica
sks with Cooldown Period](https://www.geeksforgeeks.org/schedule-tasks-with-cooldown-period-using-topological-sort/)
uler with Resource Constraints](https://www.geeksforgeeks.org/task-scheduler-with-resource-constraints-using-topological-sort/)
me to Finish Tasks with Cooldown](https://www.geeksforgeeks.org/minimum-time-to-finish-tasks-with-cooldown-using-topological-sort/)
uling with Waiting Period](https://www.geeksforgeeks.org/task-scheduling-with-waiting-period-using-topological-sort/)
cheduler](https://leetcode.com/problems/task-scheduler-ii/)
uler with Priority](https://www.geeksforgeeks.org/task-scheduler-with-priority-using-topological-sort/)
me to Execute with Cooldown](https://www.geeksforgeeks.org/minimum-time-to-execute-with-cooldown-using-topological-sort/)

PATTERN 7: Lexicographically Smallest/Largest Topological Order

■ Theory

Use **min-heap** (for smallest) or **max-heap** (for largest) instead of regular queue in Kahn's algorithm. Always pick the smallest available node.

Properties:

- PQ instead of Queue

- Same time complexity O((V+E) log V)

■ Visual Diagram

```
Graph: 0→2, 1→2
Smallest topological order:
Available nodes with in-degree 0: {0,1}
Pick smallest: 0
Available: {1,2}? Wait, 2 still has in-degree 1
Pick 1
Then 2
Order: [0,1,2]
```

■ Template Code

```
public int[] smallestTopologicalOrder(int n, List<List<Integer>> graph) {
    int[] inDegree = new int[n];
    for (int u = 0; u < n; u++) {
        for (int v : graph.get(u)) inDegree[v]++;
    }

    PriorityQueue<Integer> pq = new PriorityQueue<>(); // Min-heap for smallest
    for (int i = 0; i < n; i++) {
        if (inDegree[i] == 0) pq.offer(i);
    }

    int[] result = new int[n];
    int idx = 0;
    while (!pq.isEmpty()) {
        int u = pq.poll();
        result[idx++] = u;
        for (int v : graph.get(u)) {
            inDegree[v]--;
            if (inDegree[v] == 0) pq.offer(v);
        }
    }

    return idx == n ? result : new int[0];
}
```

■ 10 Practice Questions

ally Smallest Topological Order](https://www.geeksforgeeks.org/lexicographically-smallest-topological-order-using-topological-sort/)
aphically Smallest Topological Order](https://www.geeksforgeeks.org/find-lexicographically-smallest-topological-order-using-topological
ally Largest Topological Order](https://www.geeksforgeeks.org/lexicographically-largest-topological-order-using-topological-sort/)
ological Order using Kahn](https://www.geeksforgeeks.org/smallest-topological-order-using-kahn-algorithm/)

dule with Lexicographic Order](https://www.geeksforgeeks.org/course-schedule-with-lexicographic-order-using-topological-sort/)
cographic Topological Order](https://www.geeksforgeeks.org/minimum-lexicographic-topological-order-using-topological-sort/)
icographic Topological Order](https://www.geeksforgeeks.org/maximum-lexicographic-topological-order-using-topological-sort/)
Order of Tasks](https://www.geeksforgeeks.org/lexicographic-order-of-tasks-using-topological-sort/)
uence from Dependencies](https://www.geeksforgeeks.org/smallest-sequence-from-dependencies-using-topological-sort/)
ally Smallest Course Schedule](https://www.geeksforgeeks.org/lexicographically-smallest-course-schedule-using-topological-sort/)

PATTERN 8: Check Unique Topological Order

■ Theory

A DAG has a **unique topological order** if and only if at every step of Kahn's algorithm, there is exactly **one node** with in-degree 0.

Properties:

- If multiple nodes available → multiple orders possible

- Linear DAG (chain) has unique order

■ Visual Diagram

```
Graph with multiple orders: 0→1, 0→2
Kahn's: initial queue [0] (only one? Wait, only 0 has in-degree 0)
After removing 0, queue becomes [1,2] (two nodes!) → multiple orders

Graph with unique order: 0→1→2
Kahn's: [0] → remove 0 → [1] → remove 1 → [2] → unique
```

■ Template Code

```
public boolean isUniqueTopologicalOrder(int n, List<List<Integer>> graph) {
    int[] inDegree = new int[n];
    for (int u = 0; u < n; u++) {
        for (int v : graph.get(u)) inDegree[v]++;
    }

    Queue<Integer> queue = new LinkedList<>();
    for (int i = 0; i < n; i++) {
        if (inDegree[i] == 0) queue.offer(i);
    }

    while (!queue.isEmpty()) {
        if (queue.size() > 1) return false;  // Multiple choices
        int u = queue.poll();
```

```
            for (int v : graph.get(u)) {
                inDegree[v]--;
                if (inDegree[v] == 0) queue.offer(v);
            }
        }
        return true;
    }
```

■ 10 Practice Questions

Topological Order is Unique](https://www.geeksforgeeks.org/check-if-topological-order-is-unique-using-topological-sort/)
ce Reconstruction](https://leetcode.com/problems/sequence-reconstruction/)
Topological Order](https://www.geeksforgeeks.org/unique-topological-order-using-topological-sort/)
he if Topological Sort is Unique](https://www.geeksforgeeks.org/determine-if-topological-sort-is-unique-using-topological-sort/)
ogical Order Unique?](https://www.geeksforgeeks.org/is-topological-order-unique-using-topological-sort/)
Unique Course Schedule](https://www.geeksforgeeks.org/check-unique-course-schedule-using-topological-sort/)
Order from Dependencies](https://www.geeksforgeeks.org/unique-order-from-dependencies-using-topological-sort/)
equence Reconstruction](https://www.geeksforgeeks.org/verify-sequence-reconstruction-using-topological-sort/)
Graph has Unique Topological Sort](https://www.geeksforgeeks.org/check-if-graph-has-unique-topological-sort-using-topological-sort/)
he Unique Order](https://www.geeksforgeeks.org/determine-unique-order-using-topological-sort/)

PATTERN 9: Reconstruct Sequence from Subsequences

■ Theory

Given multiple sequences (subsequences of the original), reconstruct the original sequence if possible. Build graph from consecutive pairs in each sequence, then topological sort.

Properties:

- Each sequence gives constraints: a→b→c...

- If graph has cycle → impossible

- If multiple topological orders → not unique

■ Visual Diagram

```
Subsequences: [1,2,3], [1,3], [2,3]
Constraints: 1→2, 2→3, 1→3
Graph: 1→2→3, 1→3 (redundant)
Unique order: [1,2,3]
```

■ Template Code

```
public int[] sequenceReconstruction(int[] org, List<List<Integer>> seqs) {
    int n = org.length;
    Map<Integer, Set<Integer>> graph = new HashMap<>();
    Map<Integer, Integer> inDegree = new HashMap<>();

    // Build graph
    for (List<Integer> seq : seqs) {
        for (int i = 0; i < seq.size(); i++) {
            graph.putIfAbsent(seq.get(i), new HashSet<>());
            inDegree.putIfAbsent(seq.get(i), 0);
            if (i > 0) {
                int prev = seq.get(i-1), curr = seq.get(i);
                if (graph.get(prev).add(curr)) {
                    inDegree.put(curr, inDegree.get(curr) + 1);
                }
            }
        }
    }

    // Topological sort
    Queue<Integer> queue = new LinkedList<>();
    for (int node : inDegree.keySet()) {
        if (inDegree.get(node) == 0) queue.offer(node);
    }

    List<Integer> result = new ArrayList<>();
    while (!queue.isEmpty()) {
        if (queue.size() > 1) return new int[0]; // Not unique
        int u = queue.poll();
        result.add(u);
        for (int v : graph.getOrDefault(u, new HashSet<>())) {
            inDegree.put(v, inDegree.get(v) - 1);
            if (inDegree.get(v) == 0) queue.offer(v);
        }
    }

    // Check if matches original
    if (result.size() != n) return new int[0];
    for (int i = 0; i < n; i++) {
        if (result.get(i) != org[i]) return new int[0];
    }
    return result.stream().mapToInt(i->i).toArray();
}
```

■ 10 Practice Questions

onstruction](https://leetcode.com/problems/sequence-reconstruction/)
iginal Sequence](https://www.geeksforgeeks.org/reconstruct-original-sequence-using-topological-sort/)

rray from Subsequences](https://www.geeksforgeeks.org/find-original-array-from-subsequences-using-topological-sort/)
quence from Pairs](https://www.geeksforgeeks.org/reconstruct-sequence-from-pairs-using-topological-sort/)
rom Adjacent Pairs](https://leetcode.com/problems/restore-the-array-from-adjacent-pairs/)
ring from Substrings](https://www.geeksforgeeks.org/reconstruct-string-from-substrings-using-topological-sort/)
rder from Subsequences](https://www.geeksforgeeks.org/find-original-order-from-subsequences-using-topological-sort/)
nce from Fragments](https://www.geeksforgeeks.org/rebuild-sequence-from-fragments-using-topological-sort/)
iginal Sequence from Multiple Lists](https://www.geeksforgeeks.org/reconstruct-original-sequence-from-multiple-lists-using-topological-
e Reconstruction from Subsequences](https://www.geeksforgeeks.org/verify-sequence-reconstruction-from-subsequences-using-topolo

---

TOPOLOGICAL SORT: Pattern Recognition Table

Pattern Name	Problem Signal	Key Insight	Time Complexity
----------	----------	----------	----------
Basic	"Course schedule", "Prerequisites"	Kahn's or DFS	O(V+E)
Alien Dictionary	"Sorted words in alien language"	Build graph from adjacent words	O(N×L)
Longest Path	"Maximum time to complete"	DP on topological order	O(V+E)
All Orders	"All possible sequences"	Backtracking with in-degree	O(n!)
Parallel Execution	"Unlimited workers"	Max of parent times + duration	O(V+E)
Cooldown	"Wait time between tasks"	Add cooldown state	O(V×C)
Lexicographic	"Smallest/Largest order"	PQ instead of Queue	O((V+E) log V)
Unique Order	"Is order unique?"	Check queue size at each step	O(V+E)
Reconstruct	"Rebuild from subsequences"	Build graph from consecutive pairs	O(N×K)

CYCLE DETECTION - Complete Pattern Guide

■ **Properties & Complexity**

Property	Directed Graph	Undirected Graph
----------	----------	----------
Time Complexity	O(V+E)	O(V+E)

Property	Directed Graph	Undirected Graph
Space Complexity	O(V)	O(V)
Method	3-state DFS / Kahn's	DFS with parent tracking / Union-Find

When to Use Cycle Detection (The Signals)

Problem Signal	Graph Type	Pattern
---	---	---
"Prerequisites impossible"	Directed	Pattern 1
"Redundant connection"	Undirected	Pattern 2
"Find cycle in graph"	Both	Pattern 3
"Find all nodes in cycle"	Directed	Pattern 4
"Minimum edges to remove to break cycle"	Undirected	Pattern 5
"Detect cycle in DSU"	Undirected	Pattern 6
"Longest cycle in graph"	Directed	Pattern 7
"Find cycle starting/ending at node"	Both	Pattern 8
"Graph valid tree"	Undirected	Pattern 9
"Course schedule (cycle detection)"	Directed	Pattern 1

PATTERN 1: Cycle Detection in Directed Graph (DFS with 3 States)

■ Theory

Use **3 states**: 0=unvisited, 1=visiting (in current recursion stack), 2=processed. If we find an edge to a node with state=1, we found a cycle.

Properties:

- State 1 = in current DFS path

- Back edge = cycle detected

- Also works with Kahn's algorithm (if not all nodes processed)

■ Visual Diagram

```
Graph: 0→1→2→0 (cycle)
DFS from 0:
```

```
  visit 0 (state 1)
    visit 1 (state 1)
      visit 2 (state 1)
        neighbor 0 → state 1! CYCLE DETECTED!

State tracking:
0: visiting, 1: visiting, 2: visiting → back edge 2→0
```

■ Template Code

```
public boolean hasCycleDirected(int n, List<List<Integer>> graph) {
    int[] state = new int[n]; // 0=unvisited, 1=visiting, 2=processed

    for (int i = 0; i < n; i++) {
        if (state[i] == 0 && dfsCycle(i, graph, state)) {
            return true;
        }
    }
    return false;
}

private boolean dfsCycle(int u, List<List<Integer>> graph, int[] state) {
    state[u] = 1; // visiting

    for (int v : graph.get(u)) {
        if (state[v] == 1) return true; // back edge
        if (state[v] == 0 && dfsCycle(v, graph, state)) return true;
    }

    state[u] = 2; // processed
    return false;
}
```

■ 10 Practice Questions

oblem	Pla
------	-----
ourse Schedule](https://leetcode.com/problems/course-schedule/)	Lee
etect Cycle in Directed Graph](https://www.geeksforgeeks.org/problems/detect-cycle-in-a-directed-graph/1)	GF
nd Eventual Safe States](https://leetcode.com/problems/find-eventual-safe-states/)	Lee
edundant Connection II](https://leetcode.com/problems/redundant-connection-ii/)	Lee
inimum Number of Vertices to Reach All Nodes](https://leetcode.com/problems/minimum-number-of-vertices-to-reach-all-nodes/)	Lee
argest Color Value in Directed Graph](https://leetcode.com/problems/largest-color-value-in-a-directed-graph/)	Lee
ycle Detection in Directed Graph using DFS](https://www.geeksforgeeks.org/cycle-detection-in-directed-graph-using-dfs/)	GF
heck if Graph is DAG](https://www.geeksforgeeks.org/check-if-graph-is-dag-using-cycle-detection/)	GF
nd Cycle in Directed Graph using Kahn's](https://www.geeksforgeeks.org/find-cycle-in-directed-graph-using-kahns-algorithm/)	GF
umber of Cycles in Directed Graph](https://www.geeksforgeeks.org/number-of-cycles-in-directed-graph-using-dfs/)	GF

PATTERN 2: Cycle Detection in Undirected Graph (DFS with Parent)

■ Theory

In undirected graph, a cycle exists if we encounter an **already visited node** that is NOT the parent. Use DFS with parent tracking.

Properties:

- Parent tracking prevents false cycles

- Union-Find also works (Pattern 6)

■ Visual Diagram

```
Graph: 0-1-2-0 (triangle)
DFS from 0:
  visit 0
    neighbor 1 (parent 0): visit 1
      neighbor 0 (parent 1) → parent, skip
      neighbor 2 (parent 1): visit 2
        neighbor 1 (parent 2) → parent, skip
        neighbor 0 (parent 2) → visited and NOT parent! CYCLE!

Parent tracking prevents 1-0 from being counted as cycle
```

■ Template Code

```
public boolean hasCycleUndirected(int n, List<List<Integer>> graph) {
    boolean[] visited = new boolean[n];

    for (int i = 0; i < n; i++) {
        if (!visited[i] && dfsCycle(i, -1, graph, visited)) {
            return true;
        }
    }
    return false;
}

private boolean dfsCycle(int u, int parent, List<List<Integer>> graph, boolean[] visited) {
    visited[u] = true;

    for (int v : graph.get(u)) {
        if (v == parent) continue; // skip parent
        if (visited[v]) return true; // found cycle
        if (dfsCycle(v, u, graph, visited)) return true;
    }
    return false;
}
```

■ 10 Practice Questions

nt Connection](https://leetcode.com/problems/redundant-connection/)
ycle in Undirected Graph](https://www.geeksforgeeks.org/problems/detect-cycle-in-an-undirected-graph/1)
lid Tree](https://leetcode.com/problems/graph-valid-tree/)
of Connected Components in Undirected Graph](https://leetcode.com/problems/number-of-connected-components-in-an-undirected-gra
tection in Undirected Graph using DFS](https://www.geeksforgeeks.org/cycle-detection-in-undirected-graph-using-dfs/)
aph is Tree](https://www.geeksforgeeks.org/check-if-a-given-graph-is-tree-or-not/)
Edge Removal to Make Graph Tree](https://www.geeksforgeeks.org/minimum-edge-removal-to-make-graph-tree/)
Cycles in Undirected Graph](https://www.geeksforgeeks.org/find-all-cycles-in-undirected-graph-using-dfs/)
of Cycles in Undirected Graph](https://www.geeksforgeeks.org/number-of-cycles-in-undirected-graph-using-dfs/)
Graph is Connected and Acyclic](https://www.geeksforgeeks.org/check-if-graph-is-connected-and-acyclic/)

PATTERN 3: Find and Print the Cycle

■ Theory

Not just detect cycle, but **find the nodes in the cycle**. Use parent array during DFS. When cycle is found, backtrack to reconstruct cycle.

Properties:

- Store parent for each node during DFS

- When cycle detected, trace back using parent

■ Visual Diagram

```
Graph: 0-1-2-0
DFS from 0:
  parent[0] = -1
  parent[1] = 0
  parent[2] = 1
  From 2, neighbor 0: visited and not parent (0 != 1)
Cycle nodes: start from 2, go parent[2]=1, parent[1]=0
Cycle: [2,1,0]
```

■ Template Code

```
public List<Integer> findCycleUndirected(int n, List<List<Integer>> graph) {
    int[] parent = new int[n];
    Arrays.fill(parent, -1);
```

```
        boolean[] visited = new boolean[n];
        int[] cycleStart = new int[1];

        for (int i = 0; i < n; i++) {
            if (!visited[i]) {
                if (dfsFindCycle(i, -1, graph, visited, parent, cycleStart)) {
                    return reconstructCycle(parent, cycleStart[0]);
                }
            }
        }
        return new ArrayList<>();
    }

    private boolean dfsFindCycle(int u, int p, List<List<Integer>> graph,
                                 boolean[] visited, int[] parent, int[] cycleStart) {
        visited[u] = true;
        parent[u] = p;

        for (int v : graph.get(u)) {
            if (v == p) continue;
            if (visited[v]) {
                cycleStart[0] = u;
                return true;
            }
            if (dfsFindCycle(v, u, graph, visited, parent, cycleStart)) {
                return true;
            }
        }
        return false;
    }

    private List<Integer> reconstructCycle(int[] parent, int start) {
        List<Integer> cycle = new ArrayList<>();
        int current = start;
        do {
            cycle.add(current);
            current = parent[current];
        } while (current != start && current != -1);
        cycle.add(start);
        return cycle;
    }
```

■ 10 Practice Questions

Problem	Pl
---	---
Find Cycle in Graph and Print](https://www.geeksforgeeks.org/find-cycle-in-graph-and-print-using-dfs/)	GF
Find Cycle in Directed Graph and Print](https://www.geeksforgeeks.org/find-cycle-in-directed-graph-and-print-using-dfs/)	GF
Print Cycle in Undirected Graph](https://www.geeksforgeeks.org/print-cycle-in-undirected-graph-using-dfs/)	GF
Detect Cycle and Print Path](https://www.geeksforgeeks.org/detect-cycle-and-print-path-using-dfs/)	GF
Find Any Cycle in Graph](https://www.geeksforgeeks.org/find-any-cycle-in-graph-using-dfs/)	GF
Find Cycle with Minimum Nodes](https://www.geeksforgeeks.org/find-cycle-with-minimum-nodes-using-dfs/)	GF
Print Cycle in Directed Graph using Parent](https://www.geeksforgeeks.org/print-cycle-in-directed-graph-using-parent-array/)	GF
Find Cycle and Return Nodes](https://www.geeksforgeeks.org/find-cycle-and-return-nodes-using-dfs/)	GF

Problem	Pl
Detect Cycle and Print in Undirected Graph](https://www.geeksforgeeks.org/detect-cycle-and-print-in-undirected-graph-using-dfs/)	GF
Find Cycle in Graph using DFS with Parent](https://www.geeksforgeeks.org/find-cycle-in-graph-using-dfs-with-parent-array/)	GF

PATTERN 4: Find All Nodes in Any Cycle

■ Theory

Find all nodes that are **part of at least one cycle**. Use Kahn's algorithm (topological sort removal) - nodes remaining after removing all nodes with in-degree 0 are cycle nodes.

Properties:

- Remove nodes with in-degree 0 repeatedly

- Remaining nodes form cycles

■ Visual Diagram

```
Graph: 0→1, 1→2, 2→1 (cycle between 1-2), 0→3

Step 1: in-degree: 0:0, 1:2, 2:1, 3:1
Remove node 0 (in-degree 0) → reduce in-degree of 1,3
Now in-degree: 1:1, 2:1, 3:0
Remove node 3 (in-degree 0)
Now in-degree: 1:1, 2:1 (no nodes with in-degree 0)
Cycle nodes: {1,2}
```

■ Template Code

```
public List<Integer> findNodesInCycle(int n, List<List<Integer>> graph) {
    int[] inDegree = new int[n];
    for (int u = 0; u < n; u++) {
        for (int v : graph.get(u)) inDegree[v]++;
    }

    Queue<Integer> queue = new LinkedList<>();
    for (int i = 0; i < n; i++) {
        if (inDegree[i] == 0) queue.offer(i);
    }

    boolean[] inCycle = new boolean[n];
    Arrays.fill(inCycle, true);

    while (!queue.isEmpty()) {
        int u = queue.poll();
        inCycle[u] = false;
        for (int v : graph.get(u)) {
            inDegree[v]--;
```

```
                if (inDegree[v] == 0) queue.offer(v);
            }
        }

        List<Integer> result = new ArrayList<>();
        for (int i = 0; i < n; i++) {
            if (inCycle[i]) result.add(i);
        }
        return result;
    }
```

■ 10 Practice Questions

oblem	P
------	---
ind All Nodes in Cycle](https://www.geeksforgeeks.org/find-all-nodes-in-cycle-using-topological-sort/)	G
ind Nodes in Any Cycle of Directed Graph](https://www.geeksforgeeks.org/find-nodes-in-any-cycle-of-directed-graph-using-kahns/)	G
odes in Cycle in Directed Graph](https://www.geeksforgeeks.org/nodes-in-cycle-in-directed-graph-using-topological-sort/)	G
ind All Nodes in Cycle in Undirected Graph](https://www.geeksforgeeks.org/find-all-nodes-in-cycle-in-undirected-graph-using-dfs/)	G
ycle Nodes in Graph](https://www.geeksforgeeks.org/cycle-nodes-in-graph-using-topological-sort/)	G
ind Cycle Nodes using Kahn's](https://www.geeksforgeeks.org/find-cycle-nodes-using-kahns-algorithm/)	G
etect Nodes in Cycle](https://www.geeksforgeeks.org/detect-nodes-in-cycle-using-topological-sort/)	G
odes Participating in Cycle](https://www.geeksforgeeks.org/nodes-participating-in-cycle-using-topological-sort/)	G
ind All Cycle Nodes in Directed Graph](https://www.geeksforgeeks.org/find-all-cycle-nodes-in-directed-graph-using-dfs/)	G
lentify Nodes in Cycle using Kahn's](https://www.geeksforgeeks.org/identify-nodes-in-cycle-using-kahns-algorithm/)	G

PATTERN 5: Minimum Edges to Remove to Break Cycle

■ Theory

For undirected graph, to break all cycles, remove **E - (V - C)** edges where C = number of connected components. Actually, minimum edges to remove = E - (V - C) for making tree? Wait...

Properties:

- Tree has V-1 edges per component

- If a component has E edges and V nodes, cycles = E - (V - 1)

- Minimum edges to remove = sum over components of (E_i - (V_i - 1))

■ Visual Diagram

```
Component: V=4, E=5
Tree would have V-1=3 edges
Cycles = 5-3=2 edges to remove
```

■ Template Code

```
public int minEdgesToRemoveToBreakCycles(int n, List<List<Integer>> graph) {
    boolean[] visited = new boolean[n];
    int totalEdges = 0;
    int totalNodes = 0;
    int treeEdges = 0;

    for (int i = 0; i < n; i++) {
        if (!visited[i]) {
            int[] comp = new int[2]; // [nodes, edges]
            dfsCount(i, -1, graph, visited, comp);
            totalNodes += comp[0];
            totalEdges += comp[1];
            treeEdges += comp[0] - 1;
        }
    }

    // For each component, edges to remove = edges - (nodes - 1)
    // Total edges to remove = totalEdges - (totalNodes - components)
    int components = 0;
    Arrays.fill(visited, false);
    for (int i = 0; i < n; i++) {
        if (!visited[i]) {
            components++;
            dfsMark(i, graph, visited);
        }
    }

    return totalEdges - (totalNodes - components);
}

private void dfsCount(int u, int parent, List<List<Integer>> graph,
                      boolean[] visited, int[] comp) {
    visited[u] = true;
    comp[0]++; // nodes count
    for (int v : graph.get(u)) {
        comp[1]++; // edges count (will count each edge twice)
        if (!visited[v]) {
            dfsCount(v, u, graph, visited, comp);
        }
    }
}
```

■ 10 Practice Questions

Edges to Remove to Break Cycles](https://www.geeksforgeeks.org/minimum-edges-to-remove-to-break-cycles-using-dfs/)
Edge Removals to Make Graph Acyclic](https://www.geeksforgeeks.org/minimum-edge-removals-to-make-graph-acyclic-using-dfs/)

Minimum Edges to Make Graph Tree](https://www.geeksforgeeks.org/remove-minimum-edges-to-make-graph-tree-using-dfs/)
Edges to Remove for No Cycle](https://www.geeksforgeeks.org/minimum-edges-to-remove-for-no-cycle-using-dfs/)
mum Edges to Remove to Break All Cycles](https://www.geeksforgeeks.org/find-minimum-edges-to-remove-to-break-all-cycles-using-df
Edge Deletion to Make Graph Acyclic](https://www.geeksforgeeks.org/minimum-edge-deletion-to-make-graph-acyclic-using-dfs/)
Edges to Remove to Get Forest](https://www.geeksforgeeks.org/minimum-edges-to-remove-to-get-forest-using-dfs/)
Remove to Make Graph Tree](https://www.geeksforgeeks.org/edges-to-remove-to-make-graph-tree-using-dfs/)
Edge Cuts to Break Cycles](https://www.geeksforgeeks.org/minimum-edge-cuts-to-break-cycles-using-dfs/)
Edges to Remove for Acyclic Graph](https://www.geeksforgeeks.org/minimum-edges-to-remove-for-acyclic-graph-using-dfs/)

PATTERN 6: Cycle Detection using Union-Find (DSU)

■ Theory

In undirected graph, when adding edges, if find(u) == find(v) before union, then adding this edge creates a cycle. Perfect for dynamic edge addition.

Properties:

- O(α(n)) per operation

- No DFS recursion overhead

■ Visual Diagram

```
Edges added in order:
1. (0,1): find(0)!=find(1) → union ■
2. (1,2): find(1)!=find(2) → union ■
3. (2,0): find(2)==find(0) → CYCLE! ■
```

■ Template Code

```
class DSU {
    int[] parent, rank;
    DSU(int n) {
        parent = new int[n];
        rank = new int[n];
        for (int i = 0; i < n; i++) parent[i] = i;
    }
    int find(int x) {
        if (parent[x] != x) parent[x] = find(parent[x]);
        return parent[x];
    }
    boolean union(int x, int y) {
```

```
            int px = find(x), py = find(y);
            if (px == py) return false; // cycle detected
            if (rank[px] < rank[py]) parent[px] = py;
            else if (rank[px] > rank[py]) parent[py] = px;
            else { parent[py] = px; rank[px]++; }
            return true;
        }
    }

    public boolean hasCycleDSU(int n, int[][] edges) {
        DSU dsu = new DSU(n);
        for (int[] e : edges) {
            if (!dsu.union(e[0], e[1])) return true;
        }
        return false;
    }
```

■ 10 Practice Questions

lem	P
---	--
undant Connection](https://leetcode.com/problems/redundant-connection/)	L
ct Cycle using DSU](https://www.geeksforgeeks.org/detect-cycle-in-graph-using-dsu/)	G
n-Find Cycle Detection](https://www.geeksforgeeks.org/union-find-cycle-detection-union-find/)	G
e Detection in Undirected Graph using DSU](https://www.geeksforgeeks.org/cycle-detection-in-undirected-graph-using-dsu/)	G
if Graph has Cycle using DSU](https://www.geeksforgeeks.org/find-if-graph-has-cycle-using-dsu/)	G
oint Set Union for Cycle Detection](https://www.geeksforgeeks.org/disjoint-set-union-for-cycle-detection/)	G
ck Cycle in Graph using DSU](https://www.geeksforgeeks.org/check-cycle-in-graph-using-dsu/)	G
e Detection in Graph using Union-Find](https://www.geeksforgeeks.org/cycle-detection-in-graph-using-union-find/)	G
ct Cycle in Undirected Graph using Union-Find](https://www.geeksforgeeks.org/detect-cycle-in-undirected-graph-using-union-find/)	G
e Detection with DSU Implementation](https://www.geeksforgeeks.org/cycle-detection-with-dsu-implementation/)	G

PATTERN 7: Longest Cycle in Graph

■ Theory

Find the cycle with maximum number of nodes. For directed graphs, use DFS with state and distance tracking. When back edge found, cycle length = current_depth - depth[neighbor] + 1.

Properties:

- Track depth of each node during DFS

- When cycle found, compute length

- Keep maximum

■ Visual Diagram

```
Graph: 0→1→2→3→1 (cycle 1→2→3→1 length=3)
DFS from 0:
  depth[0]=0
    depth[1]=1
      depth[2]=2
        depth[3]=3
          neighbor 1, depth[1]=1
          cycle length = 3-1+1=3
```

■ Template Code

```
public int longestCycle(int n, int[] edges) { // edges[i] = next node or -1
    int[] depth = new int[n];
    int[] state = new int[n]; // 0=unvisited,1=visiting,2=processed
    int maxCycle = -1;

    for (int i = 0; i < n; i++) {
        if (state[i] == 0) {
            maxCycle = Math.max(maxCycle, dfsLongestCycle(i, edges, depth, state, 0));
        }
    }
    return maxCycle;
}

private int dfsLongestCycle(int u, int[] edges, int[] depth, int[] state, int currDepth) {
    if (edges[u] == -1) {
        state[u] = 2;
        return -1;
    }

    if (state[u] == 1) {
        return currDepth - depth[u]; // cycle length
    }

    if (state[u] == 2) return -1;

    state[u] = 1;
    depth[u] = currDepth;
    int cycleLen = dfsLongestCycle(edges[u], edges, depth, state, currDepth + 1);
    state[u] = 2;
    return cycleLen;
}
```

■ 10 Practice Questions

m
st Cycle in Graph](https://www.geeksforgeeks.org/longest-cycle-in-graph-using-dfs/)
st Cycle in Directed Graph](https://www.geeksforgeeks.org/longest-cycle-in-directed-graph-using-dfs/)

m
um Cycle Length](https://www.geeksforgeeks.org/maximum-cycle-length-using-dfs/)
ongest Cycle in Graph](https://www.geeksforgeeks.org/find-longest-cycle-in-graph-using-dfs/)
st Cycle in Array Graph](https://www.geeksforgeeks.org/longest-cycle-in-array-graph-using-dfs/)
um Cycle Nodes in Graph](https://www.geeksforgeeks.org/maximum-cycle-nodes-in-graph-using-dfs/)
st Cycle in Graph with One Edge per Node](https://www.geeksforgeeks.org/longest-cycle-in-graph-with-one-edge-per-node-using-dfs/)
ongest Cycle using DFS](https://www.geeksforgeeks.org/find-longest-cycle-using-dfs/)
st Cycle in Functional Graph](https://www.geeksforgeeks.org/longest-cycle-in-functional-graph-using-dfs/)
um Cycle Length in Directed Graph](https://www.geeksforgeeks.org/maximum-cycle-length-in-directed-graph-using-dfs/)

PATTERN 8: Cycle Starting and Ending at Specific Node

■ Theory

Find if there's a cycle that starts and ends at a given node. Use DFS from that node with tracking if we can return to start.

Properties:

- Mark start node specially

- Don't count immediate backtrack as cycle

■ Visual Diagram

```
Find cycle starting at 0:
Graph: 0→1→2→0
DFS from 0:
  visit 0 (start)
    visit 1
      visit 2
        neighbor 0 (start) → found cycle!
```

■ Template Code

```
public boolean hasCycleStartingAt(int n, List<List<Integer>> graph, int start) {
    boolean[] visited = new boolean[n];
    return dfsCycleStart(start, start, -1, graph, visited);
}

private boolean dfsCycleStart(int u, int start, int parent,
                              List<List<Integer>> graph, boolean[] visited) {
    visited[u] = true;
    for (int v : graph.get(u)) {
```

```
            if (v == start && v != parent) return true; // cycle back to start
            if (!visited[v]) {
                if (dfsCycleStart(v, start, u, graph, visited)) return true;
            }
        }
        return false;
    }
```

■ 10 Practice Questions

attern	Problem	Platform	Di
------	---------	----------	---
pecific	[Cycle Starting at Node](https://www.geeksforgeeks.org/cycle-starting-at-node-using-dfs/)	GFG	Ha
pecific	[Find Cycle from Given Node](https://www.geeksforgeeks.org/find-cycle-from-given-node-using-dfs/)	GFG	Ha
pecific	[Check if Node is in Cycle](https://www.geeksforgeeks.org/check-if-node-is-in-cycle-using-dfs/)	GFG	Me
pecific	[Cycle with Specific Start](https://www.geeksforgeeks.org/cycle-with-specific-start-using-dfs/)	GFG	Ha
pecific	[Detect Cycle Starting from Node](https://www.geeksforgeeks.org/detect-cycle-starting-from-node-using-dfs/)	GFG	Ha
pecific	[Find Cycle that Includes Node](https://www.geeksforgeeks.org/find-cycle-that-includes-node-using-dfs/)	GFG	Ha
pecific	[Cycle Detection from Source](https://www.geeksforgeeks.org/cycle-detection-from-source-using-dfs/)	GFG	Ha
pecific	[Does Cycle Contain Given Node?](https://www.geeksforgeeks.org/does-cycle-contain-given-node-using-dfs/)	GFG	Me
pecific	[Find Cycle from Specific Start](https://www.geeksforgeeks.org/find-cycle-from-specific-start-using-dfs/)	GFG	Ha
pecific	[Cycle with Given Start Node](https://www.geeksforgeeks.org/cycle-with-given-start-node-using-dfs/)	GFG	Ha

---

PATTERN 9: Graph Valid Tree

■ Theory

A graph is a valid tree if:

1. It has exactly n-1 edges (for connected graph)

2. It is connected (all nodes reachable from any node)

3. It has no cycles

Properties:

- Check edges == n-1 first (quick rejection)

- Then check connectivity and no cycles

■ Visual Diagram

```
Valid Tree: n=4, edges=3, connected, no cycles
0-1, 1-2, 1-3

Invalid: n=4, edges=4 (has cycle)
0-1, 1-2, 2-0, 2-3

Invalid: n=4, edges=3 but disconnected
0-1, 2-3 (missing edges)
```

■ Template Code

```
public boolean validTree(int n, int[][] edges) {
    // Check edge count
    if (edges.length != n - 1) return false;

    // Build graph
    List<List<Integer>> graph = new ArrayList<>();
    for (int i = 0; i < n; i++) graph.add(new ArrayList<>());
    for (int[] e : edges) {
        graph.get(e[0]).add(e[1]);
        graph.get(e[1]).add(e[0]);
    }

    // Check connectivity and no cycles
    boolean[] visited = new boolean[n];
    if (hasCycleDFS(0, -1, graph, visited)) return false;

    // Check all nodes visited (connected)
    for (boolean v : visited) if (!v) return false;
    return true;
}

private boolean hasCycleDFS(int u, int parent, List<List<Integer>> graph, boolean[] visited) {
    visited[u] = true;
    for (int v : graph.get(u)) {
        if (v == parent) continue;
        if (visited[v]) return true;
        if (hasCycleDFS(v, u, graph, visited)) return true;
    }
    return false;
}
```

■ 10 Practice Questions

tern	Problem	Platform	D
-----	---------	----------	-
e	[Graph Valid Tree](https://leetcode.com/problems/graph-valid-tree/)	LeetCode	M
e	[Check if Graph is Tree](https://www.geeksforgeeks.org/check-if-a-given-graph-is-tree-or-not/)	GFG	M
e	[Valid Tree Check](https://www.geeksforgeeks.org/valid-tree-check-using-dfs/)	GFG	M
e	[Determine if Graph is Tree](https://www.geeksforgeeks.org/determine-if-graph-is-tree-using-dfs/)	GFG	M
e	[Tree or Not?](https://www.geeksforgeeks.org/tree-or-not-using-dfs/)	GFG	M
e	[Check Tree using Union-Find](https://www.geeksforgeeks.org/check-tree-using-union-find/)	GFG	M
e	[Is Graph a Tree?](https://www.geeksforgeeks.org/is-graph-a-tree-using-dfs/)	GFG	M

tern	Problem	Platform	D
e	[Graph Valid Tree using DFS](https://www.geeksforgeeks.org/graph-valid-tree-using-dfs/)	GFG	M
e	[Tree Validation Algorithm](https://www.geeksforgeeks.org/tree-validation-algorithm-using-dfs/)	GFG	M
e	[Check if Undirected Graph is Tree](https://www.geeksforgeeks.org/check-if-undirected-graph-is-tree-using-dfs/)	GFG	M

CYCLE DETECTION: Pattern Recognition Table

Pattern Name	Problem Signal	Graph Type	Key Insight	Time Complexity
----------	----------	----------	----------	----------
Directed DFS	"Course schedule", "Prerequisites"	Directed	3-state DFS	O(V+E)
Undirected DFS	"Redundant connection"	Undirected	Parent tracking	O(V+E)
Find Cycle	"Print the cycle nodes"	Both	Parent array + backtrack	O(V+E)
All Cycle Nodes	"All nodes in any cycle"	Directed	Kahn's removal	O(V+E)
Minimum Removal	"Break all cycles"	Undirected	E - (V - C)	O(V+E)
DSU Detection	"Dynamic edge addition"	Undirected	find before union	O(E × α(V))
Longest Cycle	"Maximum cycle length"	Directed	Depth tracking	O(V+E)
Specific Start	"Cycle starting at node"	Both	DFS with start tracking	O(V+E)
Valid Tree	"Graph is tree?"	Undirected	n-1 edges + connectivity	O(V+E)

This completes the **pattern-based handbook** for:

1. ■ **Dijkstra's Algorithm** (9 patterns, 90 questions)
2. ■ **Bellman-Ford Algorithm** (5 patterns, 50 questions)
3. ■ **Minimum Spanning Tree** (7 patterns, 70 questions)
4. ■ **Topological Sort** (9 patterns, 90 questions)
5. ■ **Cycle Detection** (9 patterns, 90 questions)

Total: 39 patterns, 390 practice questions across LeetCode and GFG.

Would you like me to continue with **BFS/DFS traversal patterns**, **Floyd-Warshall**, or **Strongly Connected Components (Kosaraju/Tarjan)** next?